Effective DDoS Mitigation via ML-Driven In-network Traffic Shaping

Ziming Zhao, *Student Member, IEEE*, Zhuotao Liu, *Member, IEEE*, Huan Chen, Fan Zhang, *Member, IEEE*, Zhuoxue Song, and Zhaoxuan Li, *Student Member, IEEE*

Abstract—Defending against Distributed Denial of Service (DDoS) attacks is a fundamental problem in the Internet. Over the past few decades, the research and industry communities have proposed a variety of solutions, from adding incremental capabilities to the existing Internet routing stack, to clean-slate future Internet architectures, and to widely deployed commercial DDoS prevention services. Yet a recent interview with over 100 security practitioners in multiple sectors reveals that existing solutions are *still insufficient against*, due to either unenforceable protocol deployment or non-comprehensive traffic filters. This seemingly endless arms race with attackers probably means that we need a fundamental paradigm shift. In this paper, we propose a new DDoS prevention paradigm named *preference-driven and in-network enforced traffic shaping*, aiming to explore the novel DDoS prevention norms that focus on delivering victim-preferred traffic rather than consistently chasing after the DDoS attacks. Towards this end, we propose DFNet, a novel DDoS prevention system that provides reliable delivery of victim-preferred traffic *without* full knowledge of DDoS attacks. At a very high level, the core innovative design of DFNet embraces the advances in Machine Learning (ML) and new network dataplane primitives, by *encoding* the victim's traffic preference (in the form of complex ML models) into dataplane packet scheduling algorithms such that the victim-preferred traffic is forwarded with priority at line-speed, regardless of the attacker strategy. We implement a prototype of DFNet in 11,560 lines of code, and extensively evaluate it on our testbed. The results show that *a single instance of DFNet* can forward 99.93% of victim-desired traffic when facing previously unseen attacks, while imposing less than 0.1% forwarding overhead on a dataplane with 80 Gbps upstream links and a 40 Gbps bottleneck.

Index Terms—DDoS, DPDK, in-network traffic analysis, ML-based byte rule

1 Introduction

Distributed denial of service (DDoS) attacks form a serious threat to the Internet. The most recent worldwide infrastructure security report shows that DDoS attacks continue to grow in both size and frequency, where over 91% surveyed enterprises reported saturated Internet bandwidth under volumetric DDoS attacks [1]. To address this problem, the academic community has proposed a wide variety of methods, including filtering-based approaches [2], [3], capability-based approaches [4], [5], [6], overlay-based systems [7], [8], systems based on future Internet architectures [9], [10], [11], and other variants [12], [13], [14].

Yet a recent large-scale interview with more than 100 security practitioners in over ten industry sectors [15] presented several interesting observations. On the one hand, due to the heterogeneity of Autonomous Systems (ASes), academic proposals that require deployment across a large number of ASes experience limited real-world deployment. As a result, the Cloud Security Service Providers (CSSPs, *e.g.,* Cloudflare, Arbor, Akamai) play a vital role in securing today's Internet from DDoS attacks. They first redirect DDoS victims' traffic to their massively-overprovisioned data centers and then scrub such traffic using their "secret sauce" traffic filters. On the other hand, although the current CSSPs are good at filtering extremely large attacks (hundreds of gigabits per second), their empirical filters are not comprehensive enough to fully discard unwanted traffic, resulting in consistent network disruptions to downstream customers.

If the CSSPs with extraordinary operational experiences cannot invent perfect filtering rules that are effective against the entire spectrum of DDoS attacks, *it probably means that we need a fundamental paradigm shift from the network-defined filtering*. In this paper, we propose a new design for DDoS prevention named *preference-driven and in-network enforced traffic shaping*. We provide a detailed explanation of this design in § 2.2. At a very high level, we try to explore a novel DDoS prevention design that can provide reliable delivery of destination preferred traffic *without* learning the entire spectrum of DDoS attacks (we use *victim* and *destination* interchangeably). We implement this proposal by embracing the recent advances in Machine Learning (ML) and new network dataplane primitives.

In particular, the advances in ML have enabled [16], [17], [18] deep traffic analysis and pattern recognition that are more flexible and fine-grain than any linear combinations of existing traffic filters that use only coarse-grained features

- *Ziming Zhao, Huan Chen, Zhuoxue Song, and Fan Zhang are with the College of Computer Science and Technology, Zhejiang University, Hangzhou, 310027, China. Fan Zhang is also with ZJU-Hangzhou Global Scientific and Technological Innovation Center, 311200, with the Key Laboratory of Blockchain and Cyberspace Governance of Zhejiang Province, 310027, with Jiaxing Research Institute, Zhejiang University, 314000, and with Zhengzhou Xinda Institute of Advanced Technology, Zhengzhou, 450001, China. He is a visiting professor at the Singapore University of Technology & Design (SUTD). E-mail: {zhaoziming, chen_huan, fanzhang, songzhuoxue}@zju.edu.cn.*
- *Zhuotao Liu is with the Tsinghua University, Beijing 100084, China. E-mail: zhuotaoliu@tsinghua.edu.cn.*
- *Zhaoxuan Li is with the State Key Laboratory of Information Security, Institute of Information Engineering, CAS, Beijing, 100093, China, and also with the School of Cyber Security, UCAS, Beijing, 100049, China. E-mail: lizhaoxuan@iie.ac.cn.*

Manuscript received March 18, 2023.

like protocol numbers, port numbers, or packet addresses. This provides us an opportunity to accurately learn the *traffic preference* of the victim in an attack-agnostic manner (we explain why learning the pattern of attack traffic is less robust in § 2.2). Built upon this capability, we employ the new dataplane primitives that simultaneously achieve (limited) in-network programming and line-speed forwarding to *shape* the network traffic such that victim-preferred packets are forwarded with priority despite the overflowed network.

To realize the new DDoS prevention design of preference-driven and in-network enforced traffic shaping, we recognize the following three key challenges:

(i) *Stateless and real-time feature extraction.* Since DDoS attacks are intended to overflow the victim's network, the attack traffic must be discarded *in-network* at or before the bottleneck link. However, traffic features collectable at in-network points are limited. On the one hand, computing features, such as average packet sizes and packet sequencing, is difficult in real-time, and meanwhile requires the in-network points to maintain the per-flow state, facing scalability challenges. On the other hand, some features, such as backward flow information, are often invisible to the in-network points due to Internet path asymmetry.

(ii) *Achieving ML-decided traffic shaping at line-speed.* A more crucial problem is to enable the in-network points to process packets based on the inference results of ML models at line-speed. Complex ML models are often nonlinear and highly non-convex functions, or contain large numbers of decision branches, whereas the network dataplane can be viewed as a combination of (programmable) packet processing pipelines consisting of multiple tables, where each table contains a limited number of matching rules (*e.g.,* P4 [19] and OpenFlow [20]). Encoding such complex ML models only using linearly concatenated matching rules in the dataplane is therefore non-trivial.

(iii) *Mitigating the inference errors of ML models.* ML models are not silver bullets. The community has pointed out some issues with applying ML in network security [21], [22], which a key problem is the model misclassification. While we have carefully designed our learning approach to mitigate these problems, model inference errors are inevitable. Thus, it is not wise to directly discard packets that are identified as malicious. Therefore, the third challenge is how to *leverage network domain-specific designs* to offset the adversarial impacts of ML errors.

In this paper, we present DFNet, a novel DDoS prevention system in the new paradigm that handles the above challenges. DFNet is designed with three tightly coupled components named as dLearner, dEncoder, and dEnforcer.

First, dLearner is an attack-agnostic ML suite to learn the victim's traffic preference based on features directly extractable from raw packets. The goal of dLearner is not to classify each type of possible attack. Instead, it focuses on learning the pattern of traffic desired by the destination. The learning strategy in dLearner (*e.g.,* training a deep neural network or a decision tree) is adaptable based on the traffic characteristics of different destinations.

The dEncoder and dEnforcer collectively realize the complex models given by dLearner into packet scheduling algorithms in the dataplane. In particular, dEncoder is designed to encode the learned models into a list of matching rules deployable in the dataplane, while intellectually compressing the rules to avoid rule explosion. dEnforcer takes these rules as input and produces concrete traffic-shaping designs in the dataplane. The primary goal of dEnforcer is to offset the adversarial impacts of ML errors. We thus architect dEnforcer as an extensible scheduling framework centering around a packet labeling module that suggests the level of confidence that a packet is desirable based on the matching statistics. Given these labels, different types of packet schedulers can be applied within dEnforcer, and we elaborate on one concrete scheduler built upon priority queuing.

Contributions. The primary contribution of this paper is the design, implementation, and evaluation of DFNet, a novel DDoS mitigation architecture that focuses on delivering victim-preferred traffic without full knowledge of attacks. To the best of our knowledge, DFNet is the first DDoS mitigation architecture that realizes preference-driven and in-network traffic shaping, in a deployable (*e.g.,* requiring no changes to the Internet architecture), efficient (*i.e.,* imposing negligible dataplane forwarding overhead) and error-resilient (*i.e.,* robust to possible traffic preference learning errors) manner. We fully implement a prototype of DFNet in about 11,560 lines of code and evaluate it substantially on our testbed. The results show that DFNet can forward 99.93% of victim-desired traffic when facing *previously unseen DDoS attacks*, while only imposing less than 0.1% forwarding overhead on a dataplane with 80 Gbps upstream links and a 40 Gbps bottleneck.

2 Threat Model and Design Space

2.1 Threat Model

We consider strong adversaries that adopt various attack strategies. During the attack, they may control large-scale botnets so as to use different source addresses over time. Meanwhile, they might mix DDoS attacks with other types of attacks (such as infiltration-based intrusion attacks [23]). More crucially, those adversaries could adopt previously-unseen attack strategies such that the victim may not have corresponding filters in place for defense. We term all known and unknown attack strategies collectively as *full attack spectrum*.

Full DDoS Attack Spectrum. The full DDoS attack spectrum that captures all possible attack strategies is sophisticated, unknown a priori, and ever-changing. On the one hand, the known attacks are evolving over time, *e.g.,* some variants of the SYN flood attack [24]. On the other hand, the attackers consistently develop new types of attack methods, for instance, by exploiting zero-day new protocol vulnerabilities [14], [25]. Both the variants of known attacks and previously unseen attacks could cause the existing in-place filters or models to be obsolete; this problem is also known as *concept drift* [26].

2.2 DDoS Prevention in the New Paradigm

In this section, we elaborate on the two key norms of DDoS prevention techniques when facing the full DDoS

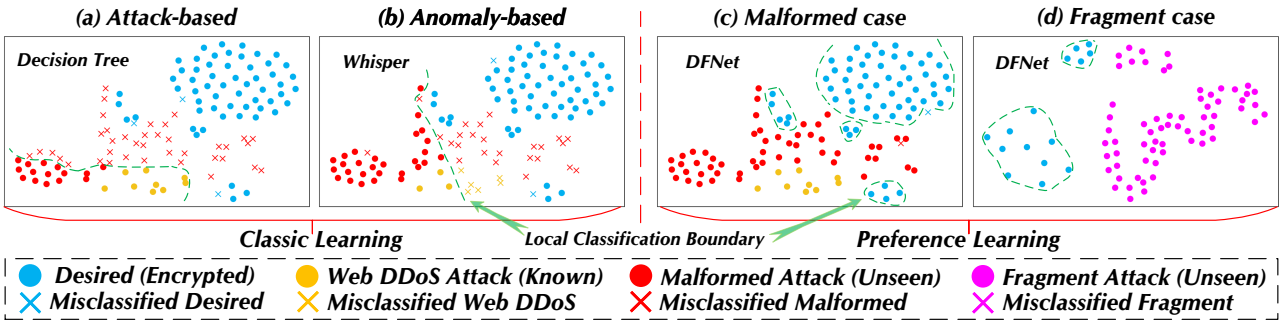

Fig. 1. Comparison of preference classifier, attack-based classifier, and anomaly-based detector.

attack spectrum, named *preference-driven* and *in-network enforced traffic shaping*, as opposed to the current paradigm of attack/anomaly-based direct filtering.

Norm 1: Preference-Driven Traffic Profiling. Given the aforementioned full spectrum of DDoS attacks, it is less likely to end the arms race by chasing after attacks with more classifiers than focusing on taking care of the victim-preferred traffic.

We clarify this observation using the following case study as shown in Figure 1. Consider a hypothetical DDoS spectrum including the common Web DDoS (*e.g.*, the HTTP GET flood, the yellow dots and forks) and two types of sophisticated attacks that have not been fully profiled by the defenders. We use the following traces to represent the sophisticated attacks: (i) the Malformed TCP Packet Attack[1] (the red dots and forks), where the adversary sends forged or defective packets (such as ACK-PSH-FIN, SYN-FIN flood), which consumes bandwidth and causes network devices to malfunction or even crash; (ii) the Fragment Attack[2] (the purple dots), where the adversary crafts a series of packet fragments that can be reassembled to bypass rule filters and launch an intrusion, *e.g.*, Teardrop, Tiny Fragment (RFC 1858), and Overlapping Fragment Attack (RFC 3128) [29], [30].

Attack-based Learning is a common type of supervised learning. Attack-based filters are trained to separate benign and attack samples in the training set. As a result, whenever the adversary adopts new attack strategies, these classifiers may become invalid. Figure 1(a) visualizes such concept drift problem using t-SNE [31]. Clearly, the attack classifier trained upon Web DDoS samples (the yellow dots) performs like random guessing when handling the previously unseen Malformed attack (the red dots and forks).

Anomaly-based Learning, a common type of unsupervised learning, proposes to train anomaly-based detectors using *only* benign samples to detect outliers. In Figure 1(b), we train an anomaly-based classifier using Whisper [32] and test its performance against the Malformed attack. We observe that the anomaly-based classifier tends to produce "inaccurate classification boundaries" (the green dashed line) that include attack samples. In fact, it could even misclassify some samples from the known Web DDoS since the anomaly-based training process cannot benefit from the known attack samples.

To tackle the above problems, we propose a new learning approach, named *preference learning*, focusing on profiling the traffic preferred by the victim and meanwhile exploiting the known attack and benign samples. Thus, the preference learning, by design, is less vulnerable to the concept drift of attack samples. Meanwhile, it tends to produce more fine-grained classification boundaries than anomaly-based designs because of the supervision or calibration by existing attack samples. We train a preference classifier using benign-preferred Classification And Regression Tree (CART, see details in § 4.1), and visualize its classification results in Figure 1(c). Under the preference classifier, the benign traffic forms several clusters, and the attack traffic (including both the known Web attack and the unseen Malformed attack) is distributed widely in the dimensionality-reduction space. Overall, the preference classifier has very few misclassifications. Figure 1(d) shows another example showing that the preference classifier is robust against the Fragment attack (the purple dots), a new type of previously unseen attack. We provide a zoomed-in view of why preference learning can classify the Malformed and Fragment attacks[3].

Concept drift of preferred traffic. The preferred traffic patterns are also dynamic and therefore the preference classifier could become outdated, too. However, the preference classifier is still superior in this case. First, the victim clearly has more instances collectable during routine operations for training the preference classifier. Yet the attack spectrum is unknown to the victim. More crucially, the network could be completely overflowed when protected by an outdated attack classifier with reduced accuracy, because the allowed volume of DDoS traffic is large enough to deny all desired traffic. Yet an outdated preference classifier still identifies the majority of benign packets (despite some false positives). Figure 2 shows a simple numerical comparison between these two types of classifiers. Given the same setting $(\mathcal{V}_D, \mathcal{V}_A, \mathcal{C})$ (indicating the desired traffic volume, attack traffic volume, and bottleneck capacity, respectively), the forwarding rate (defined as the portion of allowed desired traffic under attack) for using the preference classifier is consistently higher given the same classification accuracy. The advantage is more significant when the attack volume is higher (comparing the right subgraph to the left one).

1. For this attack type, we crawl 55 types of malformed-packet DDoS traces from the knowledge base of the DDoS defense service providers [27], [28].

2. We collect the fragment attack traffic from the Request For Comments (RFC) [29] and Wireshark sample packets [30].

3. https://github.com/Secbrain/DFNet/tree/main/preference.

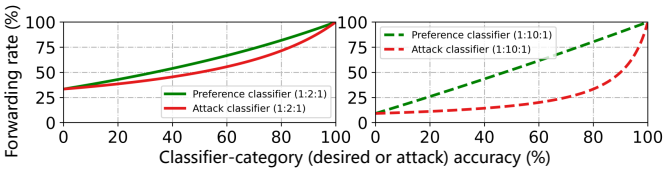

Fig. 2. Preference classifier v.s. attack classifier: effectiveness comparison under the same classification accuracy.

Caveats. Our key argument is not to show that preference learning is the "secret sauce" to defend against every type of known or unknown attacks. Rather, it tends to be more robust, compared with other approaches, when facing a large spectrum of DDoS attacks where the adversary can dynamically change its strategies. *Further, the victim is not limited to the learning method we shall present in § 4.1 to understand its traffic preference.* Any proprietary learning designs deemed effective by the victim can be used.

Norm 2: Replacing Direct Filtering with Shaping. *ML is neither a silver bullet nor the secret sauce.* On the one hand, the ML model itself can result in false inferences. On the other hand, pushing ML-powered intelligence down to the network dataplane often requires model trimming or translation (due to limited dataplane resources), which also contributes to accuracy losses. Given the inevitable inference errors, the second norm is replacing *direct filtering* with *shaping*. Traffic shaping is a conceptual idea that can be realized using various traffic scheduling designs. For instance, traffic shaping based on strict priority queuing can effectively offset the adversarial effect of packet misclassification. In particular, forwarding packets labeled as malicious (potentially including benign packets) in a low-priority service queue essentially *upgrades* the service level of these misclassified benign packets from discards (as in the direct filtering paradigm) to extra latency whenever the network has extra capacity. When suffering consistent DDoS-introduced congestion, the vast majority of malicious packets in the low-priority service queue are eventually discarded due to queue-buffer overflow, achieving the same goal as direct filtering. Thus, traffic shaping is promising to mitigate the collateral damage caused by ML imperfection.

2.3 Design Goals

Concretely, we design DFNet to achieve the following goals.

Preference-driven Traffic Prioritization. The first goal of DFNet is to mitigate the disruption of destination-desired traffic regardless of what types of attacks are being thrown at the victim, *i.e.,* the effectiveness of DFNet should not depend on the complete knowledge about the entire DDoS attack spectrum. This goal echoes back to the first norm in the new DDoS prevention paradigm. Towards this end, the way that DFNet embraces ML-powered traffic analysis is *not to* identify each individual type of attacks, as done in the existing ML literature [33], [34]. Rather, it focuses on learning the traffic pattern desired by the destination in an attack-agnostic way.

Efficient In-Network Forwarding. The usage of ML in DFNet is not limited to achieving high classification accuracy on certain datasets. Rather, DFNet's second goal is to natively embed the ML-powered traffic analysis capability into network forwarding, while simultaneously mitigating impacts of inference errors. This goal reflects the second norm in the new paradigm.

Host networking that bypasses the traditional networking stack and processes packets based on user-space applications is a promising technique. However, directly executing ML inferences as a user-space application is known to be slow on CPUs due to a large number of multiply and accumulate (MAC) operations (*e.g.,* VGG and ResNet50 need 1.4 and 3.9 billion MACs, respectively [35]). GPU acceleration is possible. Yet a recent approach [36] reports a 4.7-millisecond inference time, which can barely saturate a 1Mbps link when making the inference on each packet. Thus, DFNet proposes to approximate the complex ML models using a pipeline of matching rules to achieve nearly line-rate forwarding on common host networking platforms such as DPDK [37]. The approximation process is non-trivial. DFNet addresses this challenge by designing dEncoder to minimize the accuracy loss during the ML-to-rule translation and further using dedicated scheduling algorithms in dEnforcer to mitigate the adversarial impacts of misclassification.

Readily Deployable and Forward-Compatible. We prefer DFNet to be readily deployable in the current Internet, without incurring significant architectural modification to the Internet, requiring deployment across a wide range of ASes, or changing the client networking stack. Thus, the only external deployment required by DFNet is the software packet processing units deployed on CSSPs to execute dEnforcer (both dLearner and dEncoder are internal to the victim). Such a deployment model is an extension of CSSP's current business offering. Meanwhile, software packet processing units are commonly considered critical to enable future Internet innovation, such as supporting alternative inter-domain routing protocols in Trotsky [38] and embedding path information in path-aware Internet SCION [39]. Therefore, DFNet is not against this evolution trend.

2.4 Assumptions

We assume that the victim is able to purchase software packet processing units on CSSPs under proper commercial terms. These units, executing dEnforcer, are responsible for policing traffic on behalf of the victim to secure the downstream path from the units to the victim. Thus, the victim needs to redirect its inbound traffic sent from its clients to these units, similar to how enterprises use CSSPs today. Meanwhile, we assume that the adversary cannot control all the unit-to-victim downstream paths since otherwise it could simply discard all traffic. How to perform client load balance and client traffic migration among these units is out of the scope of this paper. This is the same underlying assumption of using CloudFlare/Akamai or any CDN-based CSSPs for DDoS mitigations. Moreover, we consider dynamic DDoS attacks in practice [40], [41], in which adversaries will deploy botnets to enable previously legitimate users to launch attacks. The evaluations in § 6.3 and § 6.4 develop the dynamic DDoS attacks in terms of attack configuration changes and fast-forwarding table spoofing. We do not assume additional collaborations from

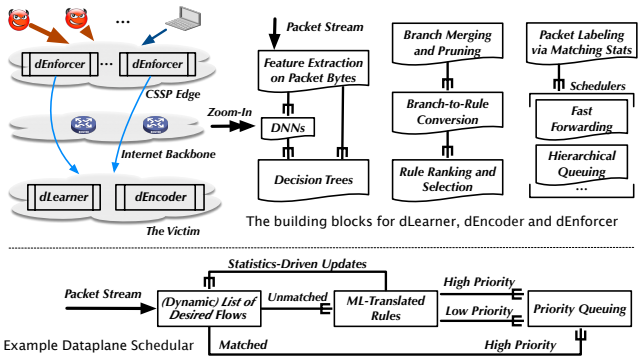

Fig. 3. The architecture of DFNet and its key components.

other Internet entities, such as clients and irrelevant ASes. In particular, we mainly focus on DDoS attack mitigation. *Slashdot effect* [42], [43], [44], [45] may be similar problems, and we discuss some potential extended solutions to deal with flash crowds in § 7.2.

3 Architecture and Overview

In Figure 3, we depict a high-level architecture of DFNet, including the key building blocks in each of its components as well as an example of dataplane scheduler. The architecture is better to read bottom-up. At the beginning, the victim employs dLearner to understand its traffic preference. For instance, the victim could collect DDoS packet streams from its CSSPs which are often exposed to various attacks. For benign samples, the victim could collect its own packet streams in routine operations while not being attacked. Depending on the characteristics of the victim's traffic and data complexity, dLearner may either train a decision tree (DT) or a deep neural network (DNN) such as Multi-Layer Perceptron (MLP). Since DNNs are nonlinear functions with a huge number of parameters, dLearner converts DNNs into decision trees through regularization (see § 4.1) to facilitate the model interpretation in dEncoder.

The raw decision trees produced by dLearner may have a large number of decision branches. Through model interpretation, we notice that naïvely allocating one matching rule for each branch is neither efficient nor necessary. dEncoder is thus designed to optimize the model-to-rule encoding process, including leaf-node pruning & merging, decision branch to rule conversion, and rule ranking (see § 4.2).

Finally, the victim deploys the rule set computed by dEncoder in dEnforcer hosted by CSSPs. The primary design challenge of dEnforcer is to handle the traffic classification errors introduced by either the imperfection of ML models or the accuracy loss during model-to-rule encoding. dEnforcer tackles this problem via flexible scheduling algorithms. We depict one example scheduler in Figure 3. The key novelty of the scheduler is that it maintains a dynamic list of desirable flows based on the statistics given by the matching rules that encode the victim's traffic preference. The scheduler allows a "premium pass" for these flows so that ML errors have zero adversarial impacts on them. A flow here aggregates the stream of packets with the same source address. We explain why this design is robust to spoofing in § 4.3.2.

4 Design Details of DFNet

4.1 dLearner Design

dLearner focuses on profiling the desired traffic that the victim hopes to protect, regardless of what attack traffic is being thrown to it. Since the models are eventually deployed at those in-network packet processing units on CSSPs, dLearner should only use features collectable at these units. For instance, it is unrealistic to use backward flow information to compute features due to Internet routing asymmetry. Further, computing features, such as packet sequences, are heavy-weight since they require these units to maintain the per-flow state. Therefore, when training models, dLearner only uses features computable from raw bytes of packets.

Consider the training dataset is collected from a specific network region, and therefore a large fraction of packets may have similar IP addresses. Directly training on such datasets may mislead the model to memorize specific IPs (or MAC, checksums, *etc.*) instead of the actual patterns. To avoid such overfitting problems, we anonymize the byte sequencing information by zeroing some bytes of the packet headers, such as identification, checksums, and the sequence number. Then we use the first N bytes (a tunable system parameter) of each packet as the training inputs. Depending on the characteristics of the victim's traffic and the complexity of the pre-collected dataset, dLearner may either train a decision tree or a Deep Neural Network (DNN) such as Multi-Layer Perceptron (MLP).

Benign-Preferred Decision Tree. Given a relatively simple pre-collected dataset containing a smaller number of attack types, dLearner can directly train a decision tree model. Particularly, based on the Classification And Regression Tree (CART), we propose a novel decision tree algorithm (called benign-preferred CART) to prefer learning the benign traffic features. Formally, denote the data at node m as Q_m, with N_m samples. According to the true labels, these samples can be divided into the attack sample set $S_a = \{x_{a_1}, x_{a_2}, \ldots, x_{a_t}\}$ and the normal sample set $S_b = \{x_{b_1}, x_{b_2}, \ldots, x_{b(N_m - t)}\}$, where t represents the number of attack samples. We can get the average of the two sets:

$$AVG_a = \frac{1}{t}\sum_{u_a \in S_a} u_a \qquad AVG_b = \frac{1}{N_m - t}\sum_{u_b \in S_b} u_b \quad (1)$$

For each candidate split $\theta = (j, t_m)$ consisting of a feature j (*i.e.*, a byte in packets) and threshold t_m, partition the data into $Q_m^{left}(\theta)$ and $Q_m^{right}(\theta)$ subsets. The quality of a candidate split of node m is then computed using an impurity (*e.g.*, Gini) function $H()$

$$G(Q_m, \theta) = \frac{N_m^{left}}{N_m} H(Q_m^{left}(\theta)) + \frac{N_m^{right}}{N_m} H(Q_m^{right}(\theta)) + e^{-\frac{|t_m - AVG_a|}{|t_m - AVG_b|}} \quad (2)$$

Where $e^{-\frac{|t_m - AVG_a|}{|t_m - AVG_b|}}$ is a penalty term that can make the split threshold t_m closer to benign traffic. Particularly, the function $y = e^{-x}$ is used to normalize the distance ratio, and it maps the range of $[0, \infty)$ to $(0, 1]$. Select the parameters that minimize the impurity

$$\theta^* = \text{argmin}_\theta \, G(Q_m, \theta) \quad (3)$$

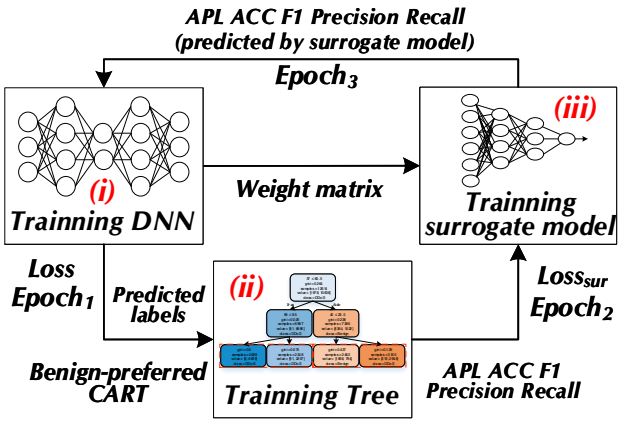

Fig. 4. The overall of the model training process.

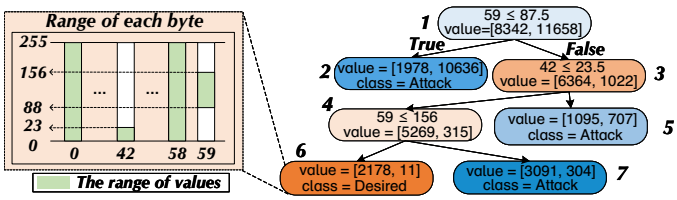

Fig. 5. Converting leaf node 6 into a byte-rule.

Recurse for subsets $Q_m^{left}(\theta^*)$ and $Q_m^{right}(\theta^*)$ until the maximum allowable depth is reached or $N_m \leq \min_{samples}$. In such a process, it takes the distance between t_m and the benign samples as a part of the quality function that can get a split threshold closer to the benign sample. To obtain a model with richer fitting capabilities, another scheme is training the deep neural network and converting it into a decision tree.

Deep Neural Network. To train the DNN and convert it into a benign-preferred decision tree, we use a surrogate model [46] to associate the DNN's weight matrix with the tree's prediction metrics (*i.e.,* Average Path Length (APL), Accuracy, Precision, Recall, and F1 score).

The overall training process is shown in Figure 4. It contains three parts: training DNN, training Tree, and training surrogate model. (i) Training DNN. Use the true labels of the dataset to train a DNN model M_{dnn}, such as MLP or GRU. The loss function is $loss$ (explain subsequently), and the number of iterations is set to $Epoch_1$. (ii) Training Tree. After training the DNN, use the predicted labels of the DNN instead of the true labels of datasets to train a Gini decision tree M_{tree}. Based on Classification And Regression Tree (CART), we improve the tree algorithm that uses a benign-preferred CART to prefer learning benign traffic features. (iii) Training Surrogate Model. The surrogate model is essentially a Multilayer Perceptron (MLP), with setting an iteration number $Epoch_2$. Input the weight matrix of DNN into surrogate model M_{sur} to get the predicted metrics by M_{sur}. Note that the outputs by the surrogate model are APL, ACC, $F1$, $Precision$, and $Recall$. Among them, the APL is expressed as follows. Assume that a combination $X = \{x_1, x_{2,\cdots}, x_n\}$ with n samples. The APL can be calculated as follows:

$$APL = \frac{1}{n}\sum_{1}^{n} deep\,(x_i) \tag{4}$$

where $deep\,(x_i)$ represents the path depth required by inputting x_i into M_{tree} to obtain the classification result. The above three processes are collectively called a conversion from DNN to Tree. After converting $Epoch_3$ times, the learning of dLearner is completed.

Loss Function. We discuss the loss functions involved in the training process.

(i) DNN Loss Function. In the training process of DNN model, we define the DNN loss function as

$$\begin{aligned} Loss = DNN_H + \lambda_{apl}\, APL_{sur} - \lambda_{acc}\, ACC_{sur} \\ - \lambda_{pre}\, Pre_{sur} - \lambda_{rec}\, Rec_{sur} - \lambda_{f1}\, F1_{sur} \end{aligned} \tag{5}$$

where DNN_H represents the crossentropy loss of DNN, APL_{sur}, ACC_{sur}, Pre_{sur}, Rec_{sur}, and $F1_{sur}$ denote Average Path Length (APL), Accuracy, Precision, Recall and F1 score of the outputs by surrogate model, respectively. Moreover, coefficient λ is the corresponding weight. These penalties in the loss function enable us to flexibly adapt the loss function based on our requirements. For instance, if the number of available matching rules in the dataplane is limited, we can properly increase λ_{apl} to produce a relatively simpler decision tree. To support the FastPath scheduler design described in § 4.3, we need to increase λ_{rec}.

(ii) Surrogate Model Loss Function. The loss function of surrogate model loss is the Mean Square Error (MSE), as follows

$$\begin{aligned} Loss_{sur} = \frac{1}{5}[(APL_{tree} - APL_{sur})^2 + (F1_{tree} - F1_{sur})^2 \\ + (ACC_{tree} - ACC_{sur})^2 + (Pre_{tree} - Pre_{sur})^2 \\ + (Rec_{tree} - Rec_{sur})^2] \end{aligned} \tag{6}$$

where APL_{tree}, ACC_{tree}, $F1_{tree}$, Pre_{tree}, and Rec_{tree} represent Average Path Length, Accuracy, Precision, Recall and F1 score of the tree model, respectively. And APL_{sur}, ACC_{sur}, $F1_{sur}$, Pre_{sur}, and Rec_{sur} represent corresponding predicted outputs by the surrogate model, respectively. Particularly, the outputs of the surrogate model will be fed to the DNN training (calculate the loss) in the next conversion process (as shown in Figure 4).

4.2 dEncoder Design

Through either of the two training methods, we can eventually get a binary decision tree model, in which each leaf node corresponds to one class and the root-to-leaf path is its decision condition. Once dLearner outputs a decision tree, a strawman design would allocate a matching rule for each leaf node in the tree. However, this is neither efficient nor necessary. On the one hand, even by employing efficient host networking platforms such as DPDK, adding matching rules in the dataplane is not free. As the number of matching rules continues to increase, the accumulated overhead of forwarding could become non-negligible. On the other hand, by analyzing the branches of the decision tree, we realize that many leaf nodes can be merged without

affecting the classification accuracy. In this section, we will elaborate on the process of encoding decision tree models into matching rules.

Pruning and Merging. A decision tree is typically terminated when the number of samples contained by the leaf node is smaller than a specified threshold. Therefore, it is possible that several leaf nodes may represent the same class and meanwhile they have a common parent node. Thus, we only need a single matching rule at the parent node to represent all these leaf nodes. By recursively traversing the tree and merging all leaf nodes that meet the above pruning conditions, we can obtain a simplified yet functionally-equivalent tree with fewer leaf nodes. Through experiments, we find that the numbers of leaves in the post-merging trees are only about one-fifth or one-third of those in the original trees.

Leaf Node to Byte-Rule Conversion. We generate a matching rule for each leaf node in the post-merging decision tree. Since the rule directly matches the raw packet bytes, we term it *byte-rule*. We use Figure 5 to illustrate the generation process. For the leaf node 6 in Figure 5, its decision branch (*i.e.,* the root-to-self path) is {59>87.5, 42≤23.5, 59≤156}, meaning that a packet is matched by this decision branch only if (i) the packet's 60-th byte (an integer within [0,255]) is greater than 88 (or equivalently in the range [88, 255]), (ii) the 43-th byte is less than 23, (iii) the 60-th byte is less than 156, and (iv) other bytes can take whatever values.

Based on the above interpretation, the leaf node to byte-rule conversion design is formulated as follows. Denote the decision branch for a leaf node as $\mathcal{P} = \{(i, \mathcal{R}_i), (j, \mathcal{R}_j), (k, \mathcal{R}_k), ...\}$ where i, j, k are the byte positions in range $[0, \mathcal{K}]$ and $\mathcal{R}_i$ specifies one range for the i-th byte position. $i, j, k...$ could be the same, indicating that the decision branch needs to satisfy multiple ranges simultaneously at the same byte position. For a given byte position $q \in [0, \mathcal{K}]$ that does not appear in $\mathcal{P}$, it indicates that the decision branch does not check this position. Then, the byte-rule for the leaf node is a list of $\mathcal{K}$ ranges and the m-th range $\mathcal{B}_m$ can be computed as

$$\mathcal{B}_m = \begin{cases} \cap\{\mathcal{R}_m\} & \text{for } \textit{all } m \text{ in } \mathcal{P} \\ [0, 255] & \text{if } m \text{ is not in } \mathcal{P} \end{cases} \tag{7}$$

Intuitively, the formulation in Equation (7) states that the m-th byte of the byte-rule should take the intersection of all the ranges appeared on m-th byte position in the decision branch $\mathcal{P}$. If $\mathcal{P}$ does not include any range for the m-th byte position, the m-th byte of the byte-rule can take arbitrary values. We depict the byte-rule for the leaf node 6 in Figure 5. As shown in the zoomed-out part on the left, the 43-th and 60-th byte positions have the range [0, 23] and [88, 156], respectively. For now, readers can assume one byte-rule is realized using one matching rule. We elaborate on the actual realization process in § 4.3.1.

An Intuitive Explanation of Byte-Rules. Each decision tree branch is converted as one byte-rule. We provide an intuitive example to explain the design of byte-rule. Assume that a victim has experienced infiltration attacks (aiming to manipulate a compromised host to access the internal system) whereas an adversary launches Slowloris DoS that has not been seen by the victim. Infiltration traffic typically sets a relatively large TTL (*e.g.,* 255) to ensure that more packets can penetrate the internal systems. Yet the benign traffic served by the victim (*e.g.,* Web services) often sets a smaller bound on TTL to avoid loops (*e.g.,* smaller than 64).

The 22-th byte of a packet represents TTL. The goal of dLearner and dEncoder is to find a value range on the 22-th byte (denoted as $\mathcal{B}_{22}$) to differentiate benign and infiltration samples (recall the victim only experienced these two types of packets). There are many feasible ranges, such as [0:100] or [0:200]. However, since we prefer to profile the features of *benign traffic*, we will choose a range that is closer to benign traffic, *e.g.,* $\mathcal{B}_{22} = [0, 64]$.

In Slowloris DoS attacks, the adversary tends to use a large TTL (*e.g.,* 128) to keep the socket connection live. Thus, even if the victim has never seen Slowloris DoS (recall this is the defining trait of sophisticated attacks), the $\mathcal{B}_{22}$, trained on benign and infiltration samples, is still effective. Of course, this is a very simplified example to explain the effectiveness behind byte-rules. Thanks to the powerful fitting capabilities of the learning techniques, it is possible to obtain a set of byte-rules that can accurately describe the raw bytes of traffic desired by the victim.

4.3 dEnforcer Design

dEnforcer is responsible for taking the matching rules given by dEncoder and generating concrete dataplane forwarding algorithms. It designs delicate scheduling algorithms to handle potential inference errors. In this section, we elaborate on the design of one such scheduler, named FastPath, that is able to *offset the adversarial effects of misclassification for a list of victim-preferred flows*.

FastPath Overview. The abstract view of FastPath is plotted in Figure 3. Its detailed design, based on DPDK [37], is depicted in Figure 6. FastPath employs two serial pipelines. The first pipeline, termed *M-pipeline*, is responsible for fast forwarding, ML-rule matching, and packet labeling based on matching results. The second pipeline, termed *Q-pipeline*, enforces priority queuing based on packet labels. Specifically, M-pipeline contains two tables: the fast forwarding table that maintains a dynamic list of flow identifiers, and the ML-rule table that holds the matching rules given by dEncoder. If a packet is matched by the fast forwarding table, it is labeled with the high priority tag and forwarded directly to the output port of the M-pipeline without traversing the ML-rule table. Otherwise, it is forwarded to the ML-rule table to first determine whether the packet is benign or malicious and then labeled accordingly (§ 4.3.1). The classification results (stored in a middle bucket) further drive the dynamic updates of the fast forwarding table (§ 4.3.2). Packets entering the Q-pipeline are forwarded based on their labels to enforce the strict priority. Low-priority packets that cannot be held in the output port buffer are discarded. Next, we discuss each module in detail.

4.3.1 ML-rule Table Design

Recall that each byte-rule given by dEncoder is a list of ranges (within [0,255]), and the i-th range specifies a range of integers that can be matched by the byte-rule on the i-th byte-position. Thus, a byte-rule with $\mathcal{K}$ ranges can include up to $256 \times \mathcal{K}$ integer values, denoted as set $\mathcal{S}$. Any packet

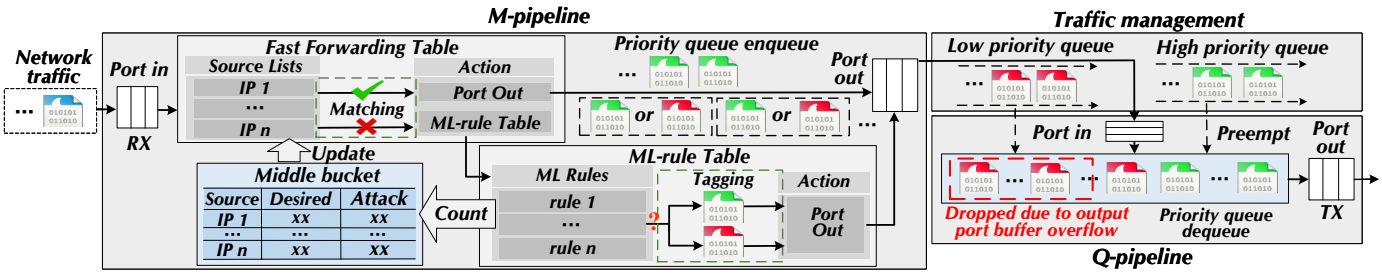

Fig. 6. The architecture of the FastPath scheduler designed to offset the adversarial impacts of possible ML inference errors.

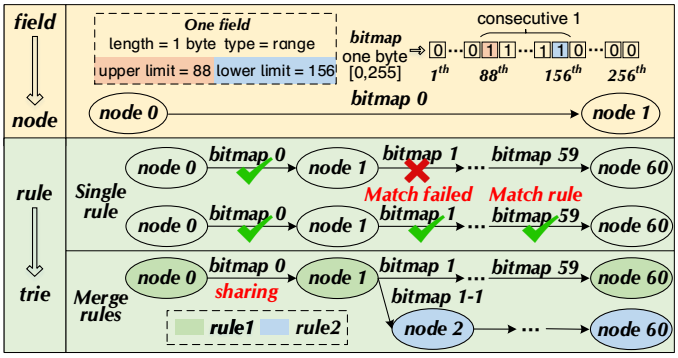

Fig. 7. The DPDK rules for realizing the 60-th range of the byte-rule.

whose first $\mathcal{K}$ bytes belong to $\mathcal{S}$ after being evaluated as a binary integer can be matched by the byte-rule. If we were using the exact match in the ML-rule table, we would need $|\mathcal{S}|$ matching rules for a single byte-rule given by dEncoder. This is obviously not scalable. To achieve range matching, we use the Access Control Library (ACL) in DPDK to implement the ML-rule table. For each byte-rule, ACL internally constructs a trie of 256-bit long bitmaps and each bitmap (node) represents one byte. Tries for different byte-rules are further merged into a large one. In other words, the number ML-rules in dataplane could be potentially smaller than the number of byte-rules. Figure 7 visualizes the DPDK rules for realizing the 60-th range of the byte-rule shown in Figure 5.

For performance reasons, the ACL requires the input field to have the *1+4X* format where the first field has to be one byte long and the length of the rest fields is dividable by 4. This imposes a restriction on the length of bytes that dLearner can use for training models. For instance, one of our experiments uses the *1+60∗1* template, indicating that dLearner extracts features from the first 60 bytes of packets. For packets that are not matched by the fast forwarding table (detailed below), they will match the rules in the ML-rule table. If a packet matches at least one rule, it is considered as benign and labeled with the high-priority tag. Otherwise, packets are forwarded to the low-priority queue.

4.3.2 Fast Forwarding Table Design

The purpose of having the fast forwarding table is to identify a dynamic list of flow identifiers such that packets matched by these identifiers can bypass the ML-rule table. As a result, ML errors will have no adversarial impacts on these flows. We adopt using source addresses as the aggregate flow identifiers. Although Internet-wide source spoofing is still possible, we have an accurate *data-driven metric* to decide whether a source address has been compromised or not. In particular, if most of packets sent by a source are classified as attack packets by the ML-rule table, the source is compromised with high probabilities. On the contrary, if an overwhelming percentage (*e.g.*, a value close to the ML model accuracy) of the packets from this source is classified as benign, it is almost certain that the adversary has not compromised this source yet. An even more desirable property of this design is that *it does not result in any collateral damages*. Specifically, even if the adversary shares the same source address with some victim-desired clients (due to NAT or the adversary is trying to mock these clients), causing this source address to be classified as being compromised, these clients can still get their packets processed by the subsequent ML-rule table, and eventually forwarded with priority.

Middle Bucket. The M-pipeline maintains a middle bucket to record the number of benign packets and attack traffic sent by different source addresses based on the statistics in the ML-rule table. When the proportion of benign packets is greater than a pre-setted threshold and meanwhile the total number of benign packets is also greater than another threshold, we will add this source address to the fast forwarding table. The two thresholds are configurable. Note that the lookup and update operations of the middle bucket are not on the critical path of the packet forwarding (implemented in a separate thread as shown in § 5). Thus, even a large middle bucket with many items will not slow down the dataplane forwarding.

Fast Forwarding. Packets arriving at the input port of the M-pipeline will first match the fast forwarding table. Packets matched by the table are labeled with the high-priority tag and forwarded directly to the output port of the M-pipeline, completely bypassing the ML-rule table. Since the fast forwarding table only matches four bytes (or 16 bytes for IPv6 addresses) whereas the ML-rule table matches tens of bytes, we can afford a larger number of rules in the fast forwarding table than the ML-rule table (§ 6).

Table Validation. The fast forwarding table needs to be validated regularly, since the adversary may suddenly start to use some of the in-table sources to launch attacks. A strong signal of this is that the number of packets with the high-priority tag received by the Q-pipeline is consistently increasing, and the average high-priority traffic volume

well surpasses the typical volume. Thus, one design to validate the fast forwarding table is gradually removing the sources originating excessive amounts of packets from the fast forwarding table, forcing their traffic to be processed by the ML-rule table. It is possible that the increase in high-priority traffic volume is indeed legitimate. Temporarily removal of these sources has limited impacts, because they will be added back to the fast forwarding table once their reputations, decided by packet statistics, are re-established. A proactive validation method, yet with higher implementation complexity, is sampling packets from all fastpath-ed flows and feeding them to the ML-rule table so as to validate these flows in nearly real-time. We use this proactive design for table validation and the detailed settings are summarized in § 6. In the cases when the high-priority queue in the Q-pipeline experiences consistent and severe congestion over a long period of time, it is reasonable to empty the fast forwarding table. This forces all packets to traverse the ML-rule table for classification. We also can develop other packet scheduling algorithms[4].

4.4 Addressing Design Concerns

Securing the Entire Downstream Path. Architecturally, DFNet is designed to protect the *entire* downstream path from the dEnforcer units (typically deployed on CSSPs, as discussed in § 2.4) to the victim. Intuitively, it is challenging to achieve this goal because DFNet has very limited information about the downstream path, such as what ASes are on the path, and where is the bottleneck.

Thanks to the priority queuing design in FastPath, DFNet ensures that the desired traffic always has advantages over the attack traffic when competing on the downstream path. We give an approximate yet easy-to-understand analysis of this advantage. Assume that the benign packet arrivals and attack packet arrivals are two independent Poisson processes with rates η_1 and η_2, respectively. During any time interval t, the number of packets that arrive at dEnforcer is $(\eta_1 + \eta_2) \cdot t$. Due to the strict priority enforcement at dEnforcer, the desired packets are always placed in front of the attack traffic despite their random arrival times. Thus, when this batch of ordered packets reaches the downstream bottleneck, as long as its service rate μ (determined by the available capacity) is greater than η_1, all desired packets will be forwarded. Therefore, without knowledge of the downstream path, DFNet secures the entire downstream path by allowing the desired traffic to *preempt* the available bandwidth. Such preemption is friendly to cross-traffic sharing the same bottleneck, since DFNet does not enforce any prioritization between the cross-traffic and DFNet-policed traffic.

Packet Reordering. Due to priority queuing, we need to explicitly address the concerns about packet reordering. Based on the design of FastPath, the senders in the current fast forwarding table are not subject to the reordering problem since all their packets are served in the same (high-priority) queue. By deploying multiple dEnforcer units on CSSPs and using source-based load balancing across these units, the victim can horizontally scale the number of fastpath-ed clients. For other desired clients, some of their packets may end up in the low-priority queue due to misclassifications. Thus, the impact of reordering is directly associated with the accuracy of ML models. We admit such a drawback of DFNet, yet argue that the benefit of delivering the vast majority of desired packets in order during DDoS attacks overwhelmingly outweighs the drawback of packet reordering for a fraction of sources.

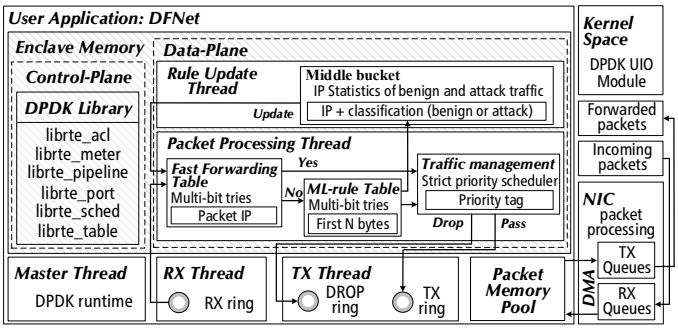

Fig. 8. The implementation of DFNet using DPDK.

5 Implementation

We implemented a prototype of DFNet in about 11,560 lines of codes (mainly in C and Python), available online[5]. Figure 8 plots the prototype architecture based on DPDK-20.11 [37]. The dataplane is multi-threaded where the forwarding thread (built using DPDK's Internet Protocol Pipeline as a building block) performs packet shaping operations (*e.g.,* matching, labeling, metering, and filtering) and the rule update thread performs statistical analysis (such as calculating benign packet ratios required by the FastPath scheduler). Eventually, DFNet is compiled and linked with the DPDK libraries specified in the control plane. We deploy DFNet as a DPDK application on our physical testbed (described in § 6). We list the main parameter settings of dLearner and dEnforcer in repository[6]. Particularly, we explain some implementation details below.

Dataplane Rule Matching. We design two types of rules (ML-based byte-rules and IP-based rules) in FastPath, using different templates that satisfy the performance optimization requirements of ACL. For IP-based rules, we use the *1+4∗1* template (whose type is bitmask matching) to match source IPs (set the offset as 26 for IPv4, and 22 for IPv6). For ML-based byte-rules, the template is selected according to the ML models. In our experiments, we use the first 60 bytes to train ML models, and therefore use the *1+60∗1* template, whose type is range matching.

Packet Labeling and Queuing. We use the rte_mbuf_sched structure in rte_mbuf (the data structure for packet buffering) for packet labeling. Specifically, based on what rules are matched by a packet, its queue_id and traffic_class (properties defined in the rte_mbuf_sched) are modified according to the actions associated with the rules. The priority queuing is built upon DPDK's traffic management module,

4. https://github.com/Secbrain/DFNet/tree/main/denforcer/.

5. Code repository https://github.com/Secbrain/DFNet/.

6. https://github.com/Secbrain/DFNet/tree/main/denforcer/.

which maintains two separate queues based on queue_id and traffic_class. Packet dequeuing enforces strict priority to ensure that high-priority packets can preempt the output buffer that is currently occupied by low-priority packets.

Traffic Metering. Measuring current bandwidth usage is important. For instance, it may trigger the fast forwarding table validation in FastPath. We build a measurement component based on DPDK's traffic metering module. We follow the trTCM algorithm [47] and define a token bucket for each packet stream we plan to collect statistics from. Since tokens are updated when packets are forwarded, we can estimate the bandwidth usage (per packet stream and aggregate) based on the number of tokens available in these buckets.

6 Evaluation

Our evaluations center around following research questions:

1) **High traffic classification accuracy.** In § 6.1, we show that DFNet achieves over ∼96% recall for various cases *where attacks are unknown a priori* using less than 100 matching rules in the dataplane.
2) **Undisputed forwarding for desired traffic.** We design a set of experiments (§ 6.2, § 6.3, § 6.4) to show that DFNet can deliver the vast majority (well above 99.9%) of victim desired traffic against dynamic DDoS attacks where the adversary can adopt new strategies previously unseen by the victim. Meanwhile, DFNet introduces less than 0.1% forwarding overhead on dataplane (§ 6.5).
3) **High scalability.** In § 6.6, we stress test DFNet to demonstrate its scalability. For the ML-rule table, DFNet scales to support $\sim 8,000$ rules before seeing throughput decline (*i.e.*, we cannot saturate the dataplane). For the fast forwarding table, a single dEnforcer instance can easily support $\sim 100,000$ rules without noticing the throughput penalty.

Testbed Setup. Our testbed has 80 Gbps-to-40 Gbps dataplane between two hosts: one serves as a packet generator and the other one deploys DFNet. The packet generator installs two 40 GbE Intel XL710-QDA2 network cards. The second host has one 40 GbE network card. Both hosts have an Intel i7-9700 CPU (3.00 GHz, 8 cores) and 16 GB memory. They are running Ubuntu 16.04.1 LTS with Linux kernel 4.15.0. We use pktgen-dpdk 20.03.0 for traffic generation.

TABLE 1
Datasets used in our evaluation.

IDS2017		DDoS2019			
Category	Sample	Category	Sample	Category	Sample
Patator	109,577	PortMap	24,012	SYN	535,587
Heartbleed	28,412	NetBIOS	464,855	NTP	23,232
Web-Attack	26,305	LDAP	50,042	DNS	29,584
Infiltration	29,881	MSSQL	624,327	SNMP	158,064
Botnet	12,853	UDP	430,061	SSDP	94,233
PortScan	162,871	UDP-Lag	338,647	WebDDoS	305,579
Benign	2,361,345	TFTP	303,777	Benign	3,527,441

Datasets. The datasets used for evaluation are summarized in Table 1, including 6 attacks in IDS2017 [23] and 13 DDoS attacks in DDoS2019 [48]. The former captures the most up-to-date attacks from 14 hosts across five OS platforms. The latter contains approximately 15-hour traffic traces of DDoS attacks. It contains not only traditional DDoS attacks but also different modern reflective ones such as PortMap, NetBIOS, etc. [48]. Both datasets use a proposed B-Profile system [23] to profile the abstract behavior of human interactions[7]. Thus, we use the traffic generated in normal scenarios (including encrypted) as benign traffic.

TABLE 2
Model-to-rule translation. {'APL': average path length; 'Leaf': node number of the raw tree; 'L-merge': node number after pruning; 'ML-rule': number of ML-rule.}

Method	Scenario	Unknown-class					
	Divide	Randomly divided			Temporally consistent		
		d_1	d_2	d_3	d_4	d_5	d_6
Tree (Benign-preferred CART)	APL	7.09	9.43	8.06	8.93	8.22	10.38
	ACC	91.21%	94.47%	95.00%	91.12%	91.41%	91.65%
	Precision	91.96%	95.05%	93.51%	91.78%	92.73%	92.52%
	Recall	90.76%	93.94%	95.98%	90.69%	90.52%	91.02%
	F1	91.36%	94.49%	94.73%	91.23%	91.61%	91.76%
	Leaf	472	664	504	584	608	704
	L-merge	128	208	164	176	160	224
	ML-rule	72	116	88	104	96	120
MLP ↓ Tree	APL	5.08	7.16	7.12	7.2	5.44	7.25
	ACC	97.44%	97.10%	98.25%	97.22%	96.92%	96.12%
	Precision	95.31%	96.01%	96.22%	96.80%	97.15%	96.31%
	Recall	**98.72%**	**98.10%**	**99.56%**	**97.50%**	**96.76%**	**95.99%**
	F1	96.99%	97.04%	97.86%	97.15%	96.95%	96.15%
	Leaf	440	576	456	442	496	524
	L-merge	104	136	136	132	128	168
	ML-rule	**56**	**80**	**72**	**69**	**64**	**86**
Whisper	ACC	97.30%	96.44%	97.77%	96.26%	96.83%	95.90%
	Precision	95.43%	95.49%	96.19%	96.37%	97.19%	96.25%
	Recall	98.42%	97.31%	98.79%	96.20%	96.58%	95.65%
	F1	96.90%	96.39%	97.48%	96.28%	96.88%	95.95%

6.1 Model-to-Rule Encoding

We evaluate the accuracy of the model-to-rule encoding mechanism. All the packets of datasets are randomly divided into two sets, denoted as Set_1 and Set_2 (there is not the same attack category in Set_1 and Set_2). For Set_1, we select S_{per} percentage attack and benign packets (denoted as Set_1^c) to simulate concept drift. Specifically, we first randomly select F_{per} percentage byte of each packet in Set_1^c and replace each of these bytes with a random integer $R_{int} \in [0, 255]$. Then, the samples in $(Set_1 - Set_1^c)$ are the training set. And the testing set includes both Set_1^c (concept drift samples) and Set_2 (unknown-class attacks). S_{per} and F_{per} are set as 20% and 30%, respectively. We perform six groups of different divisions and denote d_i as the i-th group of experiments, where $i \in [1, 6]$. The detailed packet samples of each group are shown in the online repository[8]. Note that we consider the temporal consistency [26] of dataset split in $d_4 \sim d_6$, *i.e.*, for the benign packets, the training set only from IDS2017 and the testing set only from DDoS2019.

The experimental results of the six groups are shown in Table 2 (we denote "benign" as "positive" out of the benign-preferred consideration). Method I is to train the decision tree directly with dLearner, and Method II is to train the MLP regularization to the decision tree. Both Method I and II are using benign-preferred CART (§ 4.1) to train decision trees. We find that Method II always achieves higher recall (mostly higher than 96%) and uses fewer dataplane forwarding rules (less than 90) compared with Method I.

7. Including multiple protocols (TCP, UDP, ICMP, etc.) and multiple applications (FTP, HTTP, DNS, NetBIOS, Mail, SNMP, SSH, Messenger, etc.).

8. https://github.com/Secbrain/DFNet/tree/main/evaluation.

Particularly, comparing $d_4 \sim d_6$ and $d_1 \sim d_3$, we observe that temporal consistency introduces ∼2% recall loss due to the benign sample shift. Nonetheless, we install the ML rules in d_6 to the dataplane for the remaining integration evaluations.

Compare with the recent anomaly-based detector, DFNet achieves better classification results than Whisper [32], *e.g.*, higher recall and F1 score. More importantly, DFNet has two advantages over existing ML-based schemes. (i) DFNet has dedicated designs to offset the ML imperfections. As we shall see in § 6.2, DFNet, as an integrated system, can forward over 99.9% benign traffic, higher than the standalone model accuracy. (ii) DFNet can achieve line-speed model inference over the 80 Gbps-to-40 Gbps network dataplane, while prior arts can achieve much lower forwarding rates (*e.g.*, Kitsune [16]: ∼100 Mbps and Whisper [32]: ∼10 Gbps).

TABLE 3
Forwarding rate f under classical DDoS attacks.

Pktgen rate: r	Benign traffic ratio: p				
	10%	20%	30%	40%	50%
1 × Dataplane	100%	100%	100%	100%	100%
2 × Dataplane	100%	99.98%	99.96%	99.94%	99.93%

6.2 Classical Volumetric DDoS Attacks

We start our integration test with the classical DDoS case where the attackers consistently send an excessive number of packets to overflow our network. We want to evaluate the percentage of benign traffic that is successfully forwarded. To emulate sophisticated attacks, we use the unknown-class setting with 86 ML rules installed in the dataplane (*i.e.*, d_6, the last column listed in Table 2). Since our packet generators have two 40 Gbps network cards, it can flood the 40 Gbps bottleneck with up to 2X traffic volume. We vary the expected volume of benign traffic from 10% to 50% (denoted as p) of the total traffic volume, as shown in Table 3. We count the number of benign packets arriving at the input port of dEnforcer (denoted as bp_{in}) and the number of benign packets exiting the output port (denoted as bp_{out}). Thus, the forwarding rate of the benign traffic can be computed as $f = bp_{\text{in}}/bp_{\text{out}}$. These results are shown in Table 3. It is clear that even when the attack rate doubles the total dataplane capacity, DFNet still achieves at least 99.93% forwarding rate for desired traffic.

6.3 Comparison with Current Designs

The current widely deployable DDoS prevention centers around network-defined filtering, where the CSSPs or ISPs employ scalable network filters to scrub DDoS attacks. A recent art Jaqen [49] is a representative scheme in this design space. It defines a set of policy-based defense primitives to profile previously seen attacks (*i.e.*, known attacks). It also leverages programmable switches to achieve fast policy reconfiguration. We implement a list of Jaqen policies summarized in the repository[9] using DPDK primitives (mostly based on middle buckets and blacklists).

9. https://github.com/Secbrain/DFNet/tree/main/evaluation/policies/.

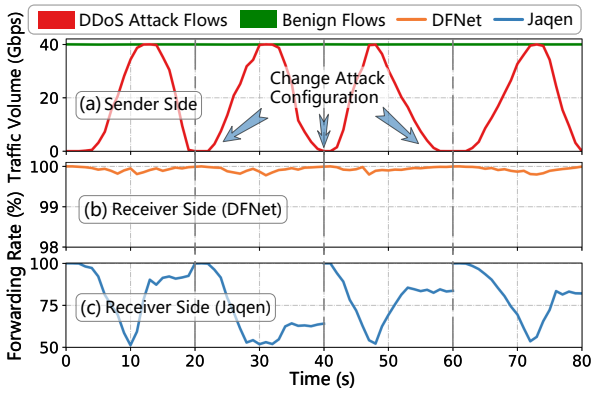

Fig. 9. The comparison of DFNet and Jaqen, a representative design in the current DDoS prevention paradigm.

We compare DFNet with Jaqen using the experiment with four attack scenes. **Scene 1**: during $t \in [0, 20]$, all DDoS attacks are previously seen by the victim (*i.e.*, known attacks); **Scene 2**: during $t \in [20, 40]$, the adversary mixes known and unknown attacks; **Scene 3**: during $t \in [40, 60]$, a set of sources first generates attack traffic and then benign traffic; **Scene 4**: For $t \in [60, 80]$, a set of sources first generates benign traffic and then attack traffic. The last two scenes emulate the cases where the adversary changes its strategies, for instance, starting to use a new set of bots and releasing the previously compromised hosts. Since Jaqen has a limited number of defense policies, all known attacks used in this experiment are selected from the row d_6 of dataset division table[10] to ensure that Jaqen has corresponding policies.

The results are shown in Figure 9. In scene 1, Jaqen starts to enforce discarding when the number of per-IP DDoS packets (identified by its predefined policies) reaches a certain threshold (also defined in policies). This often leads slow reaction in filtering. For instance, the forwarding rate of Jaqen drops significantly as attack volume rises, and starts to restore slowly at around 10s. In scene 2, Jaqen is unable to filter previously unseen attacks due to the lack of corresponding policies. Thus, its forwarding rate never restores to 100% in this scene.

In scene 3 and scene 4, if the source addresses that have sent attack traffic, Jaqen blacklists these sources and consistently blocked them even when they are sending benign packets. This shows that Jaqen may cause collateral damages to legitimate clients since its coarse-grained policies cannot differentiate benign or attack clients when they are behind the same source address (which is very common due to the wide deployment of NAT).

In contrast, DFNet consistently achieves over 99.78% forwarding rates in all scenes. *This set of experiments clearly demonstrates DFNet's advantages over the current DDoS paradigm, both in defending against previously unknown attacks and avoiding collateral damages to legitimate clients.*

We also compare the average end-to-end latency (*i.e.*, the time spent by a packet from the input port to the output port) for the dataplane baseline (DPDK layer-3 forwarding program), Jaqen, and DFNet. The results show that the base forwarding program, Jaqen, and DFNet impose about

10. https://github.com/Secbrain/DFNet/tree/main/evaluation.

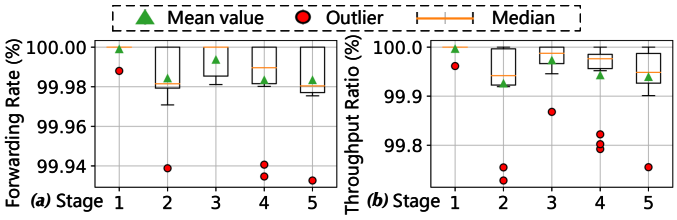

Fig. 10. Forwarding rate and throughput ratio of the benign traffic in each stage of the dynamic DDoS attacks.

36.46 μs, 36.52 μs, and 36.59 μs, respectively. It means that both DFNet and Jaqen only introduce very limited extra forwarding delay in this case (roughly 0.3%).

6.4 Adaptive Attacks

In this section, we design a case that mimics the dynamics of real-world DDoS attacks. *We intend to demonstrate the effectiveness of DFNet against the unknown spectrum of DDoS attacks. Thus, all attacks used in this experiment are unknown a priori,* i.e., *the victim never experienced these attacks previously when training dLearner.* This attack phase considers 5 stages in which the adaptive adversary deploys different attack strategies. Specifically, in **Stage 1**, only benign clients are active, and the total packet rate r is less than the dataplane capacity. In **Stage 2**, the fast forwarding table is still empty and all packets are only forwarded by the ML-rule table. The adversary starts DDoS attacks to flood our 40 Gbps dataplane. In **Stage 3**, the fast forwarding table is gradually populated. In **Stage 4**, the smart adversary changes its strategy to use the source addresses included in the fast forwarding table to launch attacks (for instance, the adversary infers some entries in the fast forwarding table or first sends benign traffic to ensure its controlled bots are included in the table). The attack volume originated from these sources is about 30% to 80% of the dataplane capacity. This triggers DFNet to start the fast forwarding table validation to purge compromised sources. In **Stage 5**, the adversary concentrates its attack power to use all sources in the fast forwarding table. This triggers DFNet to reset the fast forwarding table and force all traffic to traverse the ML-rule table. Throughout the evaluation, we use roughly 50,000 IPv4 addresses (randomly generated) and the capacity of the fast forwarding table is approximately 10,000.

In each stage, we measure three performance metrics: the forwarding rate f (as defined in § 6.2), the throughput of benign flows in aggregate and the completion time of benign packets. The results are shown in Figure 10 and Figure 11. From Figure 10(a), it is clear that the total forwarding rate of benign traffic is very close to 100%, especially in Stage 3 when the fast forwarding table is not compromised. Note that the forwarding rates in Stage 1 is 100% since the benign traffic does not saturate the dataplane.

Figure 10(b) shows the throughput ratio of input port and output port for benign traffic. We compute the input (output) port throughput by analyzing the timestamps of packets. We see more outliers in Stage 4 when the adversary starts to use FastPath-ed source IPs to launch attacks. This is expected because it takes time for DFNet to validate the fast forwarding table and purge those compromised source addresses from the table. *The throughput ratio essentially represents the forwarding overhead introduced by DFNet.* In Stage 2 and 5, when all packets are traversing the ML-rule table, DFNet introduces the highest forwarding overhead, which is roughly 0.1% (since the throughput ratio is above 99.9%).

Finally, we plot the completion time of benign packets in Figure 11. Within the time range of each subgraph, the packet generation times for both benign and attack packets are roughly uniform (except for Stage 1 which does not include attack packets). The results clearly show that the benign traffic is forwarded with much shorter completion times, demonstrating the effectiveness of DFNet for differentiating benign flows. We also plot the end-to-end forwarding latencies of individual packets on the y-axis. Comparing Stage 1 (when the network is not overflowed) with other stages, it is clear that the buffering delay dominates the end-to-end forwarding latency. This is because we configure a deep buffer at the input ports of the Q-pipeline in FastPath to avoid *input overflow*. This is consistent with CSSP's service model where they have sufficient capacity to absorb a large volume of packets. We further observe that when the fast forwarding table is active, the benign packets have the lowest forwarding latencies. For instance, in Stage 3, the latencies of benign packets drop significantly when FastPath starts to populate the fast forwarding table, whereas in Stage 5, the latencies of benign packets start to increase when DFNet is forced to reset the fast forwarding table. This is because packets matched by the fast forwarding table are directly served in the high priority queue, reducing their buffering delay.

6.5 Overhead Breakdown

To further analyze the overhead introduced by each component in the M-pipeline of FastPath (the Q-pipeline is mainly built on the priority queuing primitives without customization, and therefore we expect negligible overhead). We load varying numbers of entries in the fast forwarding table, ML-rule table and middle bucket, and then measure the packet forwarding overhead by these tables. The results are shown in Figure 12. When we measure the overhead for one component, the other two components are empty. Clearly, the processing overhead introduced by these components is negligible. For instance, with 10,000 entries, the ML-rule table introduces 0.78 μs processing time, which is three orders of magnitude smaller than the end-to-end forwarding latency (as shown in Figure 11). The fast forwarding table and middle bucket are efficient: even with 50,000 entries, their processing overhead are about 0.1 μs and 0.05 μs, respectively.

6.6 Table Capacity Evaluation

In this section, we evaluate the scalability of DFNet by studying the maximum number of rules that can be loaded in the tables of the FastPath M-pipeline, without reducing the dataplane throughput (*i.e.,* when it is impossible to saturate the 10 Gbps dataplane). As mentioned in § 4.3.1, the rules are based on different templates. Here we evaluate five rule templates: *1+4∗1* (bitmask, which is exact matching, corresponding to installing IPv4 addresses in the

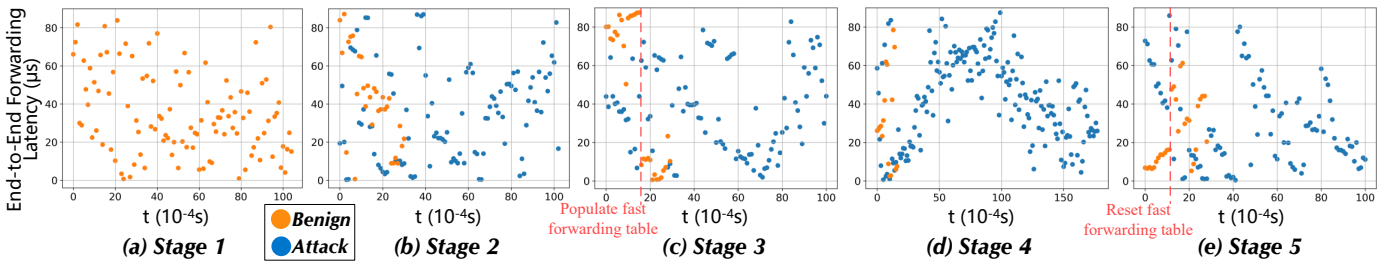

Fig. 11. The completion times and end-to-end forwarding latencies of desired and attack packets in each stage. DFNet is effective in differentiating desired flows, resulting in much shorter completion times. The end-to-end forwarding latencies of desired packets, dominated by buffering delay, are significantly lower when the fast forwarding table in FastPath is active.

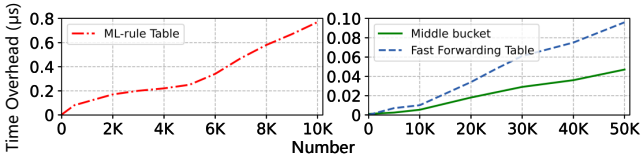

Fig. 12. The extra processing overhead introduced by each component in the M-pipeline of the FastPath scheduler.

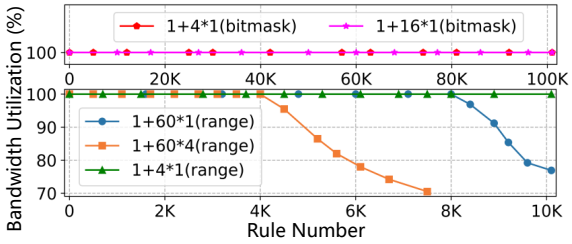

Fig. 13. The capacity of the FastPath M-pipeline.

fast forwarding table), *1+16∗1* (bitmask, corresponding to IPv6 addresses), *1+4∗1* (range matching), *1+60∗1* (range matching, corresponding to ML-rules), and *1+60∗4* (range matching).

Since our focus is to evaluate the rule capacity, the packets are randomly sampled from our PCAP traces, and the upper and lower thresholds of each byte in the rules are also randomly generated (*i.e.*, we do not care about matching accuracy in this part). The scaling results are plotted in Figure 13. For the *1+4∗1* (bitmask) template and *1+16∗1* (bitmask) template, DFNet can easily scale to 100,000 rules with negligible overhead. *This indicates that a single dEnforcer unit can support a very large fast forwarding table. As DFNet can deploy multiple dEnforcer units and employ IP-based load balancing, the total capacity for fast forwarding is horizontally scalable.* For the other templates used for ML rules, their rule limits are 10,000, 8,000, and 4,000, respectively. Note that our ML models consume no more than 100 forwarding rules, while achieving large recalls as shown in Table 2. Thus, the design of FastPath is scalable to support highly complex ML models.

7 Discussions

7.1 Deep Insights into DFNet

To provide deep insights for DFNet, we analyze the effectiveness of features from the information entropy perspective and explore the appropriate range of N.

Feature Effectiveness. We leverage the *usable information theory* [50] to conduct analysis. To measure the effectiveness of extracted information from raw data, the classic method could be using Shannon's mutual information [51]. However, important computational aspects are not considered in information theory. For example, the mutual information of original data and labels could be consistent with that of encrypted data and labels, as it allows for unbounded computation, including any needed to decrypt the text. Intuitively, when the raw data is encrypted, the information still exists but becomes unusable. In other words, mutual information can measure the amount of information, but it cannot estimate how much information can be utilized by the model. To this end, $\mathcal{V}$-*usable information* [52], [50], as the variational extension of Shannon's information theory, is presented to take into account the modeling power and computational constraints of the observer.

Specifically, let X, Y denote random variables with sample spaces $\mathcal{X}$, $\mathcal{Y}$ respectively, as well as $\emptyset$ denotes a null input that provides no information about Y. Consider a model family $\mathcal{V}$, which can be trained to map input X to its label Y, *i.e.*, $\mathcal{V} \subseteq \Omega = \{f : \mathcal{X} \cup \emptyset \rightarrow P(\mathcal{Y})\}$. Then the **predictive $\mathcal{V}$-entropy** is

$$H_{\mathcal{V}}(Y) = \inf_{f \in \mathcal{V}} \mathbb{E}[-\log_2 f[\emptyset](Y)] \tag{8}$$

and the **conditional $\mathcal{V}$-entropy** is

$$H_{\mathcal{V}}(Y|X) = \inf_{f \in \mathcal{V}} \mathbb{E}[-\log_2 f[X](Y)] \tag{9}$$

where the $\log_2$ is used to measure the entropies in bits of information. The goal is to find the $f \in \mathcal{V}$ that maximizes the log-likelihood of the label data with Eq. (9) and without Eq. (8) the input. Put succinctly, $f[X]$ and $f[\emptyset]$ produce a probability distribution over the labels. $f[\emptyset]$ models the label entropy, so $\emptyset$ can be set to an all-zero input for traffic classification tasks. Subsequently, **$\mathcal{V}$-information** refers to

$$I_{\mathcal{V}}(X \rightarrow Y) = H_{\mathcal{V}}(Y) - H_{\mathcal{V}}(Y|X) \tag{10}$$

Meanwhile, $\mathcal{V}$-information satisfies the following properties

- *Non-Negativity:* $I_{\mathcal{V}}(X \rightarrow Y) > 0$.
- *Independence:* If X is independent of Y, $I_{\mathcal{V}}(X \rightarrow Y) = 0$.

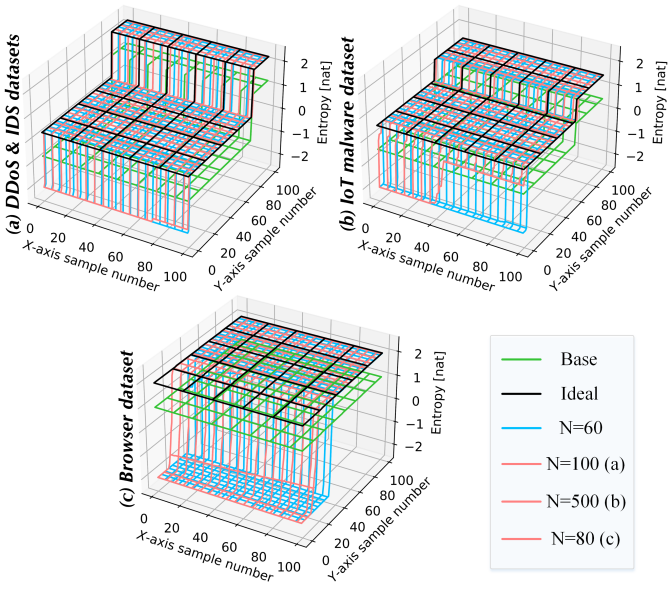

Fig. 14. The usable information entropy in various datasets.

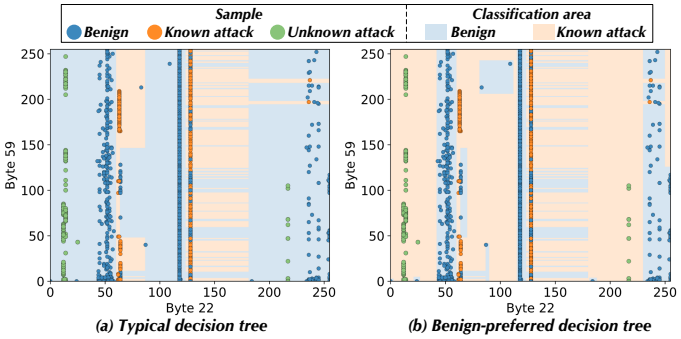

Fig. 15. The model decision boundary of benign-preferred DT.

TABLE 4
The ensemble results of decision trees, with "vote" and "match".

Tree Num.		ACC	Pre	Rec	F1	Rule
Base 1		96.12%	96.31%	95.99%	96.15%	86
Vote	10	97.21%	97.45%	97.02%	97.24%	765
	50	97.94%	98.12%	97.79%	98.00%	3,104
	100	98.37%	98.56%	98.21%	98.39%	7,988
Match	10	96.89%	96.30%	97.58%	96.94%	341
	50	97.06%	96.21%	98.03%	97.11%	1,325
	100	97.33%	95.97%	98.86%	97.39%	3,217

- *Monotonicity:* If $\mathcal{U} \in \mathcal{V}$, then $H_{\mathcal{U}}(Y) \geq H_{\mathcal{V}}(Y)$ and $H_{\mathcal{U}}(Y|X) \geq H_{\mathcal{V}}(Y|X)$.

For any instance, its usable information is expressed as the subtraction between the *predictive entropy* and the *conditional entropy*. In other words, the **pointwise $\mathcal{V}$-information** (PVI) of an instance (x, y) is

$$PVI(x \to y) = -\log_2 g[\emptyset](y) + \log_2 g'[x](y) \tag{11}$$

where $g \in \mathcal{V}$ s.t. $\mathbb{E}[-\log_2 g[\emptyset](Y)] = H_{\mathcal{V}}(Y)$ and $g' \in \mathcal{V}$ s.t. $\mathbb{E}[-\log_2 g'[X](Y)] = H_{\mathcal{V}}(Y|X)$. In fact, the relationship between PVI and $\mathcal{V}$-information is the same as between PMI (Pointwise Mutual Information) and Shannon information.

$$\begin{aligned} I(X;Y) &= \mathbb{E}_{x,y\sim P(X,Y)}[PMI(x,y)] \\ I_V(X \to Y) &= \mathbb{E}_{x,y\sim P(X,Y)}[PVI(x \to y)] \end{aligned} \tag{12}$$

Specifically, we randomly select 10K samples for various datasets, and calculate the usable information entropy based on the MLP output in dLearner. The results are shown in Figure 14, the "Base" refers to the lowest entropy value that can be correctly classified, and "Ideal" represents the entropy value that supports 100% correct predictions for every sample (*i.e.,* the ideal situation). Particularly, the stepped planes are caused by the unbalanced proportions between different categories. Among them, subfigure (a) corresponds to IDS2017 [23] & DDoS2019 [48] datasets. We can see that the usable information entropy is very close to the ideal when setting $N = 60$ (denoted as blue lines) and $N = 100$ (denoted as pink lines), although there are a few misclassifications. Therefore, setting $N = 60$ is sufficient on these datasets given that most samples already have large usable information entropy values. For subfigure (b), it refers to the IoT malware dataset [53], we observe the model has a lot of misclassifications when $N = 60$, as shown by many blue lines being lower than the green. As a comparison, when $N = 500$ (denoted as pink lines), the usable information entropy value is larger and the accuracy is improved. In Figure 14 (c), we perform the four-category classification task (*i.e.,* Chrome, Firefox, Samsung Internet, and UC) based on the browser dataset [54]. It is clear that the model exhibits a large accuracy loss when $N = 60$ because the TCP field may not be enough to identify browser fingerprints. However, when $N = 80$ (denoted as pink lines), the usable information entropy value significantly increases due to some SSL/TLS fields (*e.g., content type*, *version*) being included.

Overall, in situations such as DDoS detection, even using the first 60 bytes, there is enough information to support classification. Actually, as N increases, more information is included and performance could be further improved (§ 7.2). Moreover, for the choice of N, on the one hand, we can according to the specific task, *e.g.,* TCP fields are not enough for identity browser fingerprints so that more bytes such as SSL/TLS fields could be considered. On the other hand, we can calculate the usable information entropy based on the above method to search and determine the suitable N setting.

Benign-Preferred and Typical DT. To provide deep insights into the benign-preferred decision tree, we depict and visualize model classification boundaries in Figure 15, based on group d_6 of § 6.1. It is displayed in two dimensions: $\mathcal{B}_{22}$ and $\mathcal{B}_{59}$, the former corresponds to the time to live (TTL) field and the latter mainly about the TCP timestamp value. In Figure 15 (b), we observe that proposed benign-preferred DT tends to tighter borders for benign instances as so enables to portray desired traffic and isolate unknown attacks (denoted as green dots). However, the typical decision tree in Figure 15 (a) produces a wider boundary for benign so that unseen attacks are also considered legitimate. These observations echo our original intention of designing the benign-preferred decision tree in § 4.1.

Tree Ensemble in DFNet. Furthermore, readers could be

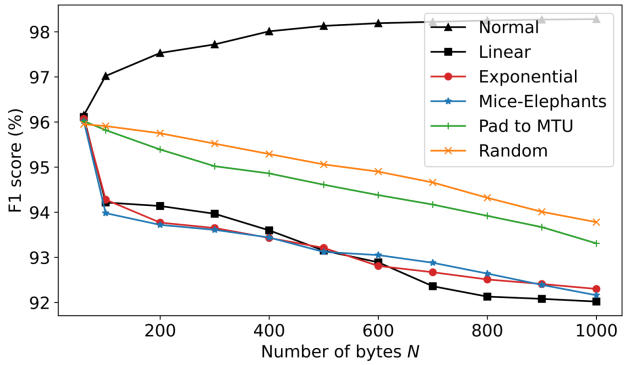

Fig. 16. The F1 score with various padding-based adversaries.

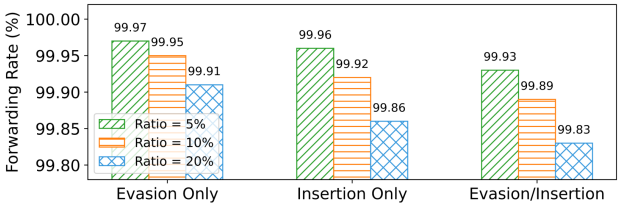

Fig. 17. The forwarding rate against evasion/insertion packets with diverse ratios.

interested in the ensemble method of decision trees. Based on the group d_6 of § 6.1, we develop two kinds of ensemble methods for dLearner. For one thing, the typical voting strategy is employed, *i.e.,* counting the results of test samples in each tree, and the prediction depends on the majority. For another, consider the priority for legitimate instances, the second strategy refers to labeling a sample as benign as long as it matches the benign rule of any tree. These two strategies are denoted "Vote" and "Match" respectively in Table 4. In the "Vote" strategy, we find that as the number of trees increases, the model performance indeed improves with respect to classification metrics, followed by a linear increase in the number of rules. As for the "Match" strategy, there is a significant improvement in recall (notably, "benign" refers to the "positive") but a slight decrease in precision because of the one-vote benign scheme. While, in this situation, the number of rules can be greatly reduced since there is no need to count the number of matched rules and these rules can be merged with each other.

7.2 Security of DFNet

Bottleneck Capacity. DFNet is a DDoS defense design that aims to deliver the victim-preferred traffic over DDoS-introduced congested bottleneck links. DFNet itself does not increase the bottleneck capacity. Thus, packet losses due to the lack of bandwidth are out of scope. Particularly, sometimes unpopular Websites instantly become extremely popular after being mentioned in a popular news feed, also called the *Slashdot effect* [42], [43], [44], [45]. Such flash crowds are somewhat similar to DDoS attacks, increasing the bandwidth bottleneck may be one way, we also discuss potential solutions with DFNet. For one thing, we can set a larger buffer size in the dataplane to process the requests when there is free bandwidth. For another, given that Slashdot mainly contains massive similar traffic in a short period of time, we can slightly modify the pipeline to cope with it. One solution is to count the packets matched by each byte rule since the traffic hit by the same rule may have similar behaviors. In this way, when the network is overflowed for a long time, the rule template with a larger number of hits can be set to discard first.

Adversarial Attack against DFNet. We evaluate here the robustness of DFNet against adversarial attacks. Due to the complexity of network interactions, attackers cannot arbitrarily change traffic content like other tasks such as image classification. For example, the attackers need to guarantee the tampered sample could pass the flow check (*i.e.,* IP Checksum) and can preserve the original malicious functions. We consider two kinds of practical/representative attacks from previous work.

(i) *Packet Padding.* To obfuscate traffic, Dyer *et al.* [55] propose to perform packet padding which is available in practice even if the payload will be encrypted. Specifically, there are five schemes. *Linear padding*: all packet lengths are increased to the nearest multiple of 128, or the Maximum Transmission Unit (MTU), whichever is smaller. *Exponential padding*: all packet lengths are increased to the nearest power of two, or the MTU, whichever is smaller. *Mice-Elephants padding*: if the packet length is ≤ 128, then the packet is increased to 128 bytes; otherwise it is padded to the MTU. *Pad to MTU*: all packet lengths are increased to the MTU. *Random MTU padding*: let M be the MTU and L be the input packet length, for each packet, a value $r \in \{0, 8, 16, \ldots, M - L\}$ is sampled uniformly at random and the packet length is increased by r. As shown in Figure 16, dLearner's F1 score only changes slightly regardless of which padding strategies when $N = 60$. As N increases, the F1 score of dLearner improves a bit since more byte information is considered (echoes back to § 7.1), but it tends to be less robust for packet padding. This is because the byte information following the header fields can be exploited, while they are also pronely tampered with by a padding-based adversary. Nonetheless, the F1 is still greater than 92%, and the priority queue design of dEnforcer in DFNet can help alleviate the impact of misclassification.

(ii) *Evasion/Insertion Attacks.* The most recent research [56] presents that using Selective Symbolic Execution (S2E) to analyze the TCP implementation of OS kernel enables producing *insertion* and *evasion* packets to elude inspection. As another kind of attack, we construct the insertion and evasion packets with various ratios. The forwarding ratio results of DFNet are summarized in Figure 17, we conduct 9 groups of experiments with the diverse ratios of insertion and evasion, *i.e.,* $\{5\%, 10\%, 20\%\}$. The observation refers that the impact of the insertion packet is greater than the evasion packet. Overall, the forwarding rate will decrease as the ratio of evasion/insertion packets increases given it will affect the per-packet identification result and the benign ratio of IP in the fast forwarding table. Thanks to the dynamic update of table validation, the impact of cumulative errors is alleviated, resulting in the forwarding rate of DFNet always being higher than 99.8%.

7.3 Practicality of DFNet

Real-world Deployment. There are multiple deployment models for DFNet in practice. So far, we have primarily

considered deploying DFNet on CSSPs. One benefit of this deployment model is that many hops on the downstream paths (from dEnforcer units to the victim) may potentially locate inside the private network of the CSSPs. This reduces the probability that an adversary gains control over the downstream paths so that it could completely discard all traffic. Another deployment model is that the victim can deploy DFNet on its ISPs. This is promising since ISPs are gradually entering the market of security services [15]. Note that DFNet deployed at the victim's ISP cannot prevent DDoS attacks trying to disconnect the victim's ISP from its upstream ISPs. However, the victim's ISP, now a victim itself, can independently deploy DFNet on its upstream ISPs. *The neat idea of DFNet is that it does not require the cooperation from unrelated parties. Rather, independent deployment is sufficient.*

Some Variation Designs of DFNet. In this paper, we mainly focus on characterizing benign traffic and detecting known/unknown attacks. While in practice, there can also be some adjustments and variant designs. For example, if attack traffic is common, or practitioners have more prior knowledge of attack data, we can profile attack traffic, generate corresponding byte rules, and construct a blacklist on the data plane to filter the attack traffic. In addition, considering the dynamic nature of traffic and the diversity of source IPs, more priority levels can be maintained. One possible solution is to set IPs that are always benign as high priority, set sources that are constantly shifting between benign and malicious as medium priority (they may have backdoors/vulnerabilities), and set addresses that send DDoS traffic as low priority.

Enhanced Internet Capabilities. The research community has proposed various designs to fundamentally make the Internet more secure, such as preventing IP spoofing and securing BGP routing. Although DFNet is designed for the status-quo Internet, it can also greatly benefit from any enhanced Internet capabilities. For instance, with continuous progress on reducing spoofable source addresses [57], FastPath can mainly focus on spoofable sources during fast forwarding table validation and implement the early drop table without collateral damages on victim-desired clients. Further, embedding DFNet into the future Internet architectures (*e.g.,* SCION [9]) is another interesting direction.

Prior work [58] use Trusted Executing Environment to achieve verifiable network filtering, yet lacking detailed billing and dispute resolution designs. We leave the exploration to future work.

Caveats and Limitations. We recognize the following caveats and limitations of DFNet. (i) Preference-driven learning is not meant to defend against every type of known or unknown attack. Like all other learning or pattern-recognition based approaches, preference learning also have false classifications (part of our future work is to further harden the design of preference learning to reduce false classifications). Yet, we argue that preference learning in principle is more robust, compared with other approaches, when facing a large spectrum of DDoS attacks where the adversary could dynamically change their attack strategies. (ii) The lack of real network traces in evaluations. Our current evaluations use datasets (although commonly used in other related literature) that are generated in lab or controlled networks. This may reduce the learning complexity. We are working on to harden evaluations with other datasets.

8 Related Work

DDoS Defense. DDoS mitigation is a profound research problem. The community has designed many approaches, roughly categorized into filtering-based approaches [2], [59], [3], capability-based approaches [4], [5], [6], overlay-based systems [7], [8], systems based on future Internet architectures [9], [10], [11], and other variants [12], [13], [60]. The shared challenges among these proposals, as confirmed by a large-scale interview with $\sim$100 industrial security experts [15], are deployability and efficiency against previously unseen attacks. DFNet *simultaneously* addresses both challenges by realizing data-driven traffic shaping (based on victim's traffic preference) at CSSPs, while introducing negligible overhead in the dataplane.

Meanwhile, a series of software-defined network (SDN)-based solutions [61], [62], [63] are proposed given its decoupling of data plane and control plane, programmability, *etc.* For example, Hu *et al.* [64] suggest a traffic migration mechanism, and Rifai *et al.* [65] present to compress and reduce the number of rules, which may be combined with DFNet. Different from previous work, DFNet embraces advanced models and bridges ML with rules. More importantly, DFNet implements line-speed traffic shaping on the dataplane to mitigate the impact of misclassification or realize customized pipelines.

ML Usage in Encrypted Traffic Detection. Some works using ML to detect encrypted traffic are also related to this paper. These approaches can be generally categorized into statistical-based methods [66], [67], [68], sequence-based approaches [69], [70], [32], and others [71], [18], [72] that use DNNs to detect traffic. Since these works require complex feature extraction and a long-time prediction, they cannot be deployed in the real network and predicted in real-time. However, DFNet can perform traffic detection well and achieve traffic filtering while meeting bandwidth requirements.

Hardware Primitives. There is a line of research regarding realizing ML models or complex security policies beyond filtering rules inside the network via the support of new hardware primitives, such as programmable switches [73], [14], [49]. Although switch ports tend to have higher capacity (*i.e.,* 100 Gbps), translating complex and non-linear deep models into switch chips with limited memory and computation capability (*e.g.,* the lack of support for float number computation) is still an open problem. *Thus, none of these approaches have concrete dataplane designs that can execute complex ML models.* Further exploration of hardware assisted forwarding in DFNet is part of our future work.

9 Conclusion

In this paper, we propose DFNet, a novel DDoS prevention architecture that focuses on providing reliable protection for victim-desired traffic via preference-driven in-network traffic shaping. At its core, DFNet encodes the victim's traffic preference (expressed in the form of complex ML

models) into dataplane scheduling algorithms such that victim-desired traffic is forwarded with priority at line-speed, regardless of the attack strategies. We implement a prototype of DFNet and evaluate it extensively on our testbed. The results show that DFNet can forward over 99.93% of victim-desired traffic in various attack scenarios where the adversary may launch previously unseen attacks, while only imposing less than 0.1% forwarding overhead on the network dataplane.

ACKNOWLEDGMENTS

We are grateful to the reviewers for their very constructive feedback and insightful comments.

REFERENCES

[1] NETSCOUT, "Worldwide Infrastructure Security Report," https://www.netscout.com/sites/default/files/2019-03/SECR_005_EN-1901\OT1\textendashWISR.pdf, 2019.
[2] S. Savage, D. Wetherall, A. Karlin, and T. Anderson, "Practical Network Support for IP Traceback," in *ACM SIGCOMM*, 2000.
[3] X. Liu, X. Yang, and Y. Lu, "To Filter or to Authorize: Network-Layer DoS Defense Against Multimillion-node Botnets," in *ACM SIGCOMM*, 2008.
[4] A. Yaar, A. Perrig, and D. Song, "SIFF: A Stateless Internet Flow Filter to Mitigate DDoS Flooding Attacks," in *IEEE S&P*, 2004.
[5] X. Liu, X. Yang, and Y. Xia, "NetFence: Preventing Internet Denial of Service from Inside Out," in *ACM SIGCOMM*, 2011.
[6] Z. Liu, H. Jin, Y.-C. Hu, and M. Bailey, "MiddlePolice: Toward Enforcing Destination-Defined Policies in the Middle of the Internet," in *ACM CCS*, 2016.
[7] C. Dixon, T. E. Anderson, and A. Krishnamurthy, "Phalanx: Withstanding Multimillion-Node Botnets," in *USENIX NSDI*, 2008.
[8] A. D. Keromytis, V. Misra, and D. Rubenstein, "SOS: Secure Overlay Services," in *ACM SIGCOMM*, 2002.
[9] X. Zhang *et al.*, "SCION: Scalability, Control, and Isolation on Next-Generation Networks," in *IEEE S&P*, 2011.
[10] D. G. Andersen *et al.*, "Accountable Internet Protocol (AIP)," in *ACM SIGCOMM*, 2008.
[11] D. Naylor *et al.*, "XIA: Architecting a More Trustworthy and Evolvable Internet," *ACM SIGCOMM CCR*, 2014.
[12] M. Walfish, M. Vutukuru, H. Balakrishnan, D. Karger, and S. Shenker, "DDoS Defense by Offense," in *ACM SIGCOMM*, 2006.
[13] Y. Gilad, A. Herzberg, M. Sudkovitch, and M. Goberman, "CDN-on-Demand: An Affordable DDoS Defense via Untrusted Clouds," *NDSS*, 2016.
[14] M. Zhang *et al.*, "Poseidon: Mitigating volumetric ddos attacks with programmable switches," in *NDSS*. The Internet Society, 2020.
[15] Z. Liu, H. Jin, Y.-C. Hu, and M. Bailey, "Practical proactive DDoS-attack mitigation via endpoint-driven in-network traffic control," *IEEE/ACM Transactions on Networking*, 2018.
[16] Y. Mirsky *et al.*, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in *NDSS*. The Internet Society, 2018.
[17] D. Barradas *et al.*, "FlowLens: Enabling Efficient Flow Classification for ML-based Network Security Applications," 2021.
[18] C. Xu, J. Shen, and X. Du, "A Method of Few-Shot Network Intrusion Detection Based on Meta-Learning Framework," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3540–3552, 2020.
[19] P. Bosshart *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, 2014.
[20] N. McKeown *et al.*, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, 2008.
[21] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
[22] D. Arp *et al.*, "Dos and Don'ts of Machine Learning in Computer Security," in *USENIX Security Symposium*, 2022.
[23] C. I. for Cybersecurity, "Intrusion detectionevaluation dataset (cicids2017)." [EB/OL], 2018, https://www.unb.ca/cic/datasets/ids-2017.html Accessed November 27, 2020.
[24] L. Zhu *et al.*, "Connection-oriented DNS to improve privacy and security," in *IEEE Symposium on Security and Privacy*, 2015, pp. 171–186.
[25] M. Antonakakis *et al.*, "Understanding the mirai botnet," in *USENIX Security Symposium*, 2017, pp. 1093–1110.
[26] F. Pendlebury *et al.*, "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time," in *USENIX Security Symposium*, 2019, pp. 729–746.
[27] MazeBolt, "DDoS Defense Knowledge Base," https://kb.mazebolt.com/knowledgebase/.
[28] H3C, "Attack detection and prevention configuration," http://www.h3c.com/en/d_202012/1371121_294551_0.htm.
[29] R. Editor, "RFC Documents," https://datatracker.ietf.org/doc/.
[30] Wireshark, "Wireshark Sample Captures," https://wiki.wireshark.org/SampleCaptures.
[31] L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.
[32] C. Fu *et al.*, "Realtime Robust Malicious Traffic Detection via Frequency Domain Analysis," in *CCS*. ACM, 2021, pp. 3431–3446.
[33] A. Praseed and P. S. Thilagam, "Modelling Behavioural Dynamics for Asymmetric Application Layer DDoS Detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 617–626, 2021.
[34] M. E. Ahmed, S. Ullah, and H. Kim, "Statistical Application Fingerprinting for DDoS Attack Mitigation," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 6, pp. 1471–1484, 2019.
[35] V. Sze *et al.*, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, 2017.
[36] A. Dhakal and K. Ramakrishnan, "NetML: An NFV Platform with Efficient Support for Machine Learning Applications," in *IEEE Conference on Network Softwarization (NetSoft)*, 2019.
[37] Intel, "Data Plane Development Kit," http://www.dpdk.org, 2014.
[38] J. McCauley, Y. Harchol, A. Panda, B. Raghavan, and S. Shenker, "Enabling a permanent revolution in internet architecture," in *ACM SIGCOMM*, 2019.
[39] A. Perrig *et al.*, *SCION: a secure Internet architecture*. Springer, 2017.
[40] A. Wang, W. Chang, S. Chen, and A. Mohaisen, "A Data-Driven Study of DDoS Attacks and Their Dynamics," *IEEE Trans. Dependable Secur. Comput.*, vol. 17, no. 3, pp. 648–661, 2020.
[41] A. Wang, A. Mohaisen, W. Chang, and S. Chen, "Capturing ddos attack dynamics behind the scenes," in *DIMVA*, ser. Lecture Notes in Computer Science, vol. 9148. Springer, 2015, pp. 205–215.
[42] J. Jung, B. Krishnamurthy, and M. Rabinovich, "Flash crowds and denial of service attacks: characterization and implications for cdns and web sites," in *WWW*. ACM, 2002, pp. 293–304.
[43] I. Ari, B. Hong, E. L. Miller, S. A. Brandt, and D. D. E. Long, "Managing flash crowds on the internet," in *MASCOTS*. IEEE Computer Society, 2003, pp. 246–249.
[44] T. Stading, P. Maniatis, and M. Baker, "Peer-to-peer caching schemes to address flash crowds," in *IPTPS*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 203–213.
[45] A. A. Ahmed, A. Jantan, and T. C. Wan, "Filtration model for the detection of malicious traffic in large-scale networks," *Comput. Commun.*, vol. 82, pp. 59–70, 2016.
[46] M. Wu, M. C. Hughes, S. Parbhoo, M. Zazzi, V. Roth, and F. Doshi-Velez, "Beyond Sparsity: Tree Regularization of Deep Models for Interpretability," in *AAAI*. AAAI Press, 2018, pp. 1670–1678.
[47] J. Heinanen and R. Guérin, "A Two Rate Three Color Marker," RFC 2698, September, Tech. Rep., 1999.
[48] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.
[49] Z. Liu *et al.*, "Jaqen: A High-Performance Switch-Native Approach for Detecting and Mitigating Volumetric DDoS Attacks with Programmable Switches," in *USENIX Security Symposium*, 2021, pp. 3829–3846.
[50] Y. Xu, S. Zhao, J. Song, R. Stewart, and S. Ermon, "A theory of usable information under computational constraints," in *ICLR*. OpenReview.net, 2020.
[51] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.

[52] K. Ethayarajh, Y. Choi, and S. Swayamdipta, "Understanding dataset difficulty with *V*-usable information," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 5988–6008.
[53] "A labeled dataset with malicious and benign IoT network traffic," https://www.stratosphereips.org/datasets-iot23.
[54] T. van Ede *et al.*, "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *NDSS*. The Internet Society, 2020.
[55] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2012, pp. 332–346.
[56] Z. Wang *et al.*, "SymTCP: Eluding Stateful Deep Packet Inspection with Automated Discrepancy Discovery," in *NDSS*. The Internet Society, 2020.
[57] M. Luckie, R. Beverly, R. Koga, K. Keys, J. A. Kroll, and K. Claffy, "Network hygiene, incentives, and regulation: deployment of source address validation in the Internet," in *ACM SIGSAC Conference on Computer and Communications Security*, 2019.
[58] D. Gong *et al.*, "Practical verifiable in-network filtering for DDoS defense," in *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2019.
[59] J. Ioannidis and S. M. Bellovin, "Implementing Pushback: Router-Based Defense Against DDoS Attacks," in *USENIX NSDI*, 2002.
[60] S. K. Fayaz, Y. Tobioka, V. Sekar, and M. Bailey, "Bohatei: Flexible and Elastic DDoS Defense," in *USENIX Security Symposium*, 2015.
[61] B. Alhijawi *et al.*, "A survey on dos/ddos mitigation techniques in sdns: Classification, comparison, solutions, testing tools and datasets," *Comput. Electr. Eng.*, vol. 99, p. 107706, 2022.
[62] R. Sahay *et al.*, "Aroma: An SDN based autonomic ddos mitigation framework," *Comput. Secur.*, vol. 70, pp. 482–499, 2017.
[63] N. Ahuja *et al.*, "Automated DDOS attack detection in software defined networking," *J. Netw. Comput. Appl.*, vol. 187, p. 103108, 2021.
[64] D. Hu *et al.*, "FADM: ddos flooding attack detection and mitigation system in software-defined networking," in *GLOBECOM*. IEEE, 2017, pp. 1–7.
[65] M. Rifai *et al.*, "Too many SDN rules? compress them with MINNIE," in *GLOBECOM*. IEEE, 2015, pp. 1–7.
[66] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust Smartphone App Identification via Encrypted Network Traffic Analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 1, pp. 63–78, 2018.
[67] F. Liu, J. Guo, X. Huang, and J. C. S. Lui, "eBA: Efficient Bandwidth Guarantee Under Traffic Variability in Datacenters," *IEEE/ACM Trans. Netw.*, vol. 25, no. 1, pp. 506–519, 2017.
[68] J. David and C. Thomas, "Efficient DDoS flood attack detection using dynamic thresholding on flow-based network traffic," *Comput. Secur.*, vol. 82, pp. 284–295, 2019.
[69] Z. Zhao *et al.*, "ERNN: Error-Resilient RNN for Encrypted Traffic Detection towards Network-Induced Phenomena," *IEEE Transactions on Dependable and Secure Computing*, 2023.
[70] Z. Song *et al.*, "I2RNN: An Incremental and Interpretable Recurrent Neural Network for Encrypted Traffic Classification," *IEEE Transactions on Dependable and Secure Computing*, 2023.
[71] Z. Zhao *et al.*, "CMD: Co-analyzed IoT Malware Detection and Forensics via Network and Hardware Domains," *IEEE Transactions on Mobile Computing*, 2023.
[72] X. Han *et al.*, "Network intrusion detection based on n-gram frequency and time-aware transformer," *Comput. Secur.*, vol. 128, p. 103171, 2023.
[73] J. Xing *et al.*, "Netwarden: Mitigating network covert channels while preserving performance," in *29th USENIX Security Symposium*, 2020, pp. 2039–2056.

Ziming Zhao is a Ph.D. student in Zhejiang University, Hangzhou, China. He has published more than 5 papers in international journals and conference proceedings, including TIFS, TDSC, TMC, ESE, CCS, and AAAI. His research interests include machine learning, traffic identification, AI security, and privacy-preserving.

Zhuotao Liu (Member, IEEE) received the BS degree from Shanghai Jiao Tong University, China, and the PhD degree from the University of Illinois at UrbanaChampaign, Champaign, Illinois. He is currently an assistant professor of Institute for Network Sciences and Cyberspace, Tsinghua University, China. Before joining Tsinghua, he was a technical lead at Google, managing massive-scale software-defined datacenter networks. His research interests include network security & privacy, Blockchain infrastructure, datacenter networking and systems security.

Huan Chen received the B.S. degree in College of Computer Science and Technology, Zhejiang University, Hangzhou, China. She is currently working toward the Ph.D. degree in the security of cyberspace at Zhejiang University. Her research interests include machine learning and traffic identification.

Fan Zhang (Member, IEEE) received his Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, CT, USA, in 2011. He is currently a Full Professor with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China, and also with the Alibaba–Zhejiang University Joint Institute of Frontier Technologies, Hangzhou. His research interests include system security, hardware security, network security, cryptography, and computer architecture.

Zhuoxue Song received the B.S. degree in computer science and technology from Hangzhou Dianzi University, in 2021. He is currently working toward the M.S. degree in the security of cyberspace at Zhejiang University, Hangzhou, China. His main research interests include encrypted traffic classification and intrusion detection.

Zhaoxuan Li (Student Member, IEEE) is a Ph.D. student in State Key Laboratory of Information Security (SKLOIS), Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), Beijing, China. He has published more than 10 papers in international journals and conference proceedings, including TIFS, TDSC, TMC, ESE, COMNETS, and ICWS. His research interests include blockchain security, formal methods, and traffic identification.